



WSQ · SOFTWARE DESIGN LEVEL 3

AI Vibe Coding with C#

Course Code: TGS-2023039178

Tertiary Infotech Academy Pte Ltd · UEN 201200696W

v7.0 · 13 September 2026 · Trainer: Dr Alfred Ang



Digital Attendance (Mandatory)

1

AM / PM

Record both daily sessions using the current QR supplied by the trainer.

2

Assessment attendance

Record the assessment session before beginning the papers.

3

QR identity

Use the assigned attendance record and verify the submitted session.

4

Course portal

The trainer directs the current SSG/TRAQOM attendance workflow through the LMS.

About the Trainer



General Trainer

Facilitator template

NAME / ROLE

QUALIFICATIONS

INDUSTRY EXPERTISE

TRAINING / CONTACT

About the Trainer



Dr Alfred Ang

Technology educator and course
developer

BACKGROUND

Software engineering, AI and cloud computing

QUALIFICATIONS

PhD · ACTA · DACE (source credentials)

TEACHING FOCUS

C# component design, debugging and evidence

CONTACT

enquiry@tertiaryinfotech.com

Learner Introduction

1

Work context

Name a software task from your workplace.

2

Existing skills

Describe your experience with variables, methods and tools.

3

Design goal

Choose one component you want to build.

4

Evidence goal

Explain how you will know the component works.

Ground Rules

1

Participate

Ask questions and explain your decisions.

2

Respect

Listen to different approaches and review code constructively.

3

Focus

Keep phones silent; step out quietly when necessary.

4

AI review

Use synthetic inputs; inspect and test every proposed change.

Lesson Plan — Day 1

Time	Teaching / activity block	Hours
09:00–11:00	Topic 1 · Activities 1–2	2
11:00–13:00	Topic 2 · first part	2
13:00–14:00	Lunch (excluded)	1
14:00–15:00	Topic 2 · completion · Activities 3–4	1
15:00–18:00	Topic 3 · Activities 5–6	3

WHEN IT MATTERS 8 contact hours; Topic 1 = 1 h classroom + 1 h practical. Topics 2–3 each = 1.5 h classroom + 1.5 h practical.

Lesson Plan — Day 2

Time	Teaching / assessment block	Hours
09:00–12:00	Topic 4 · Activities 7–8	3
12:00–13:00	Topic 5 · first part	1
13:00–14:00	Lunch (excluded)	1
14:00–16:00	Topic 5 · completion · Activities 9–10	2
16:00–17:00	Written Assessment · 5 SAQs	1
17:00–18:00	Practical Performance · 5 tasks	1

WHEN IT MATTERS Approved total: 14 h instruction (7 classroom + 7 practical) and 2 h assessment. No separate break is counted as instruction.

Learning Outcomes

1

LO1 · K1 / A1

Determine basic software components using C# methodologies to meet functional specifications.

2

LO2 · K2 / A2

Apply C# methodologies and tools for software creation.

3

LO3 · K3 / A3

Select essential C# controls and features to meet software design requirements.

4

LO4 · K4 / A4

Examine the interoperability and functionality of C# software components.

5

LO5 · K5 / A5

Generate C# design documentation aligned with user specifications.

Briefing for Assessment

1

Individual work

No discussion, photographs or recording of papers.

2

Open book

Use course slides, Learner Guide and approved materials.

3

Duration

WA: 60 min · PP: 60 min; answer all five items in each.

4

Submission

Complete the provided document, submit on LMS and sign the assessment summary.

Assessment and Funding

1

WA · Knowledge

Five open-ended questions: K1–K5.

2

PP · Ability

Five practical tasks: A1–A5.

3

Attendance

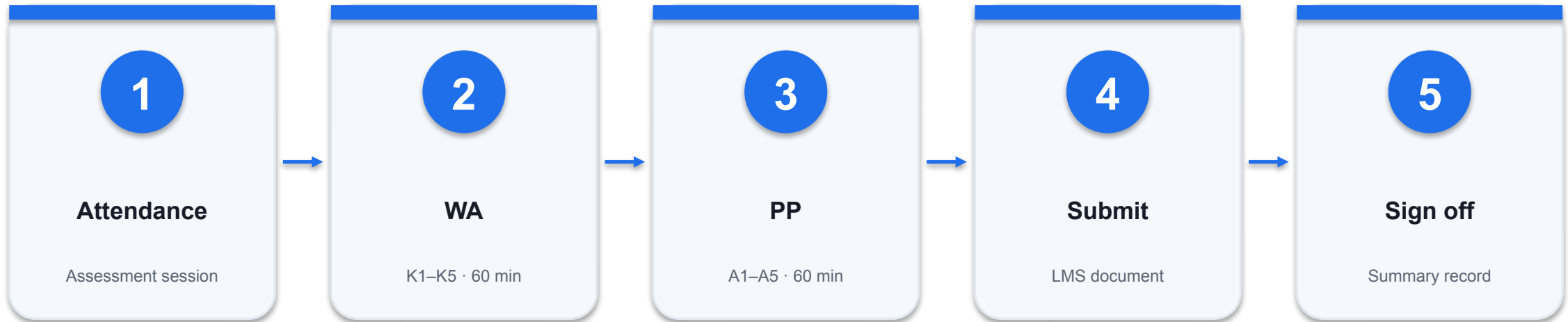
At least 75% attendance and competency in prescribed assessments.

4

Appeal

Discuss the outcome with the assessor; use the provider's published appeal process.

Assessment Flow



EVIDENCE PATH

WA then PP, followed by submission, assessor feedback and signed assessment summary.

Download Course Material

Course record

- Sign in to the LMS and open this enrolled course.

<https://lms-tms.tertiaryinfotech.com/>

Material panel

- Trainer slides, learner PDF, LG, LP and activities appear as separate resources.

Assessment panel

- Use the current WA and PP candidate documents during assessment.

Access the C# Activities

1

GitHub repository

One folder per activity, with C# source and project files.

2

Prompt PDF

Each activity includes printable vibe-coding prompts.

3

Runnable baseline

Expected outputs and executable acceptance checks.

4

Detailed guidance

Follow the LG and activity README for procedures.

<https://github.com/tertiarycourses/TGS-2023039178-AI-Vibe-Coding-with-C->

TOPIC 1 · K1 / A1

AI-assisted C# component design

Determine basic software components using C# methodologies to meet functional specifications.

Human review checks the component contract

Candidate: reject price < 0

Boundary test: price = 0 must fail.

Repair: reject price ≤ 0 .
Human review + actual execution
establish the result.



Imagegen concept illustration; labels and rule are editable.

Source, assembly and runtime boundaries

Topic 1 · concrete C# mechanism and observable result

```
// Program.cs -> compiler -> assembly -> CLR
Console.WriteLine(Environment.Version);
Console.WriteLine(typeof(string).Assembly.GetName().Name);
```

OBSERVATION

C# compiles into a managed assembly; the runtime loads its referenced libraries.

INPUT / TYPE

// Program.cs -> compiler -> assembly -> CLR

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K1 / A1; apply to Activities 1–2.

Source, assembly and runtime boundaries — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	C# compiles into a managed assembly; the runtime loads its referenced libraries.	Check the stated input/type and observed result.
Failure path	An installed runtime without an SDK cannot build source.	Reproduce the smallest input that exposes this defect.
Design control	Use modern .NET for cross-platform examples; .NET Framework is a distinct legacy Windows product.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

A project is a reproducible compiler contract

Topic 1 · concrete C# mechanism and observable result

```
<TargetFramework>net10.0</TargetFramework>  
<Nullable>enable</Nullable>  
<TreatWarningsAsErrors>true</  
TreatWarningsAsErrors>
```

OBSERVATION

The project fixes the API target and makes nullable warnings visible.

INPUT / TYPE

```
<TargetFramework>net10.0</TargetFramework>
```

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K1 / A1; apply to Activities 1–2.

A project is a reproducible compiler contract — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	The project fixes the API target and makes nullable warnings visible.	Check the stated input/type and observed result.
Failure path	An older SDK cannot compile a newer target framework.	Reproduce the smallest input that exposes this defect.
Design control	Keep project settings under version control; never accept an AI retargeting change without reviewing its impact.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

One console entry point

Topic 1 · concrete C# mechanism and observable result

```
using System;  
Console.WriteLine("Hello, warehouse!");  
// Top-level statements generate the entry  
point.
```

OBSERVATION

The executable starts once and prints a deterministic message.

INPUT / TYPE

using System;

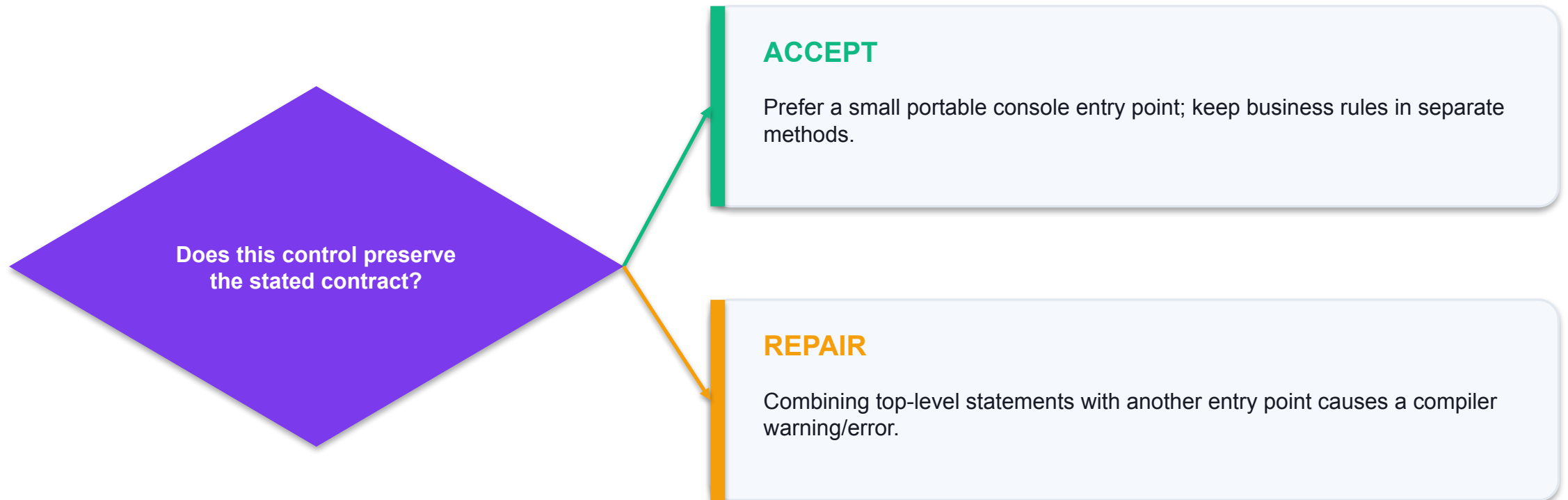
DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K1 / A1; apply to Activities 1–2.

One console entry point — design decision



Observable evidence: The executable starts once and prints a deterministic message.

Identifiers and keyword escaping

Topic 1 · concrete C# mechanism and observable result

```
int _stock = 10;  
string productName = "Rice";  
string @class = "Food";  
Console.WriteLine($"{productName}: {_stock}");
```

OBSERVATION

Underscore may start an identifier; @ escapes a keyword identifier.

INPUT / TYPE

```
int _stock = 10;
```

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K1 / A1; apply to Activities 1–2.

Identifiers and keyword escaping — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	Underscore may start an identifier; @ escapes a keyword identifier.	Check the stated input/type and observed result.
Failure path	Names starting with a digit or containing a space are invalid.	Reproduce the smallest input that exposes this defect.
Design control	Use PascalCase for public members and camelCase for locals; prefer clear names to escaped keywords.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

A functional requirement becomes an invariant

Topic 1 · concrete C# mechanism and observable result

```
string? error = Validate("Rice",0m,1,"Food");  
// R2: price must be strictly greater than 0.  
// Expected: "price"; no record is created.
```

OBSERVATION

A precise acceptance example exposes the forbidden price boundary.

INPUT / TYPE

```
string? error = Validate("Rice",0m,1,"Food");
```

DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K1 / A1; apply to Activities 1–2.

A functional requirement becomes an invariant — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	A precise acceptance example exposes the forbidden price boundary.	Check the stated input/type and observed result.
Failure path	A vague requirement such as good validation has no testable pass condition.	Reproduce the smallest input that exposes this defect.
Design control	Give each requirement an ID, concrete inputs and a measurable expected result.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Separate business rules from user interface controls

Topic 1 · concrete C# mechanism and observable result

```
// View collects text -> parser -> Product  
constructor.  
// Product enforces Price > 0 and Stock >= 0.  
// View renders success only after Add  
succeeds.
```

OBSERVATION

The same invariant survives replacing a console view with a GUI.

INPUT / TYPE

// View collects text -> parser -> Product constructor.

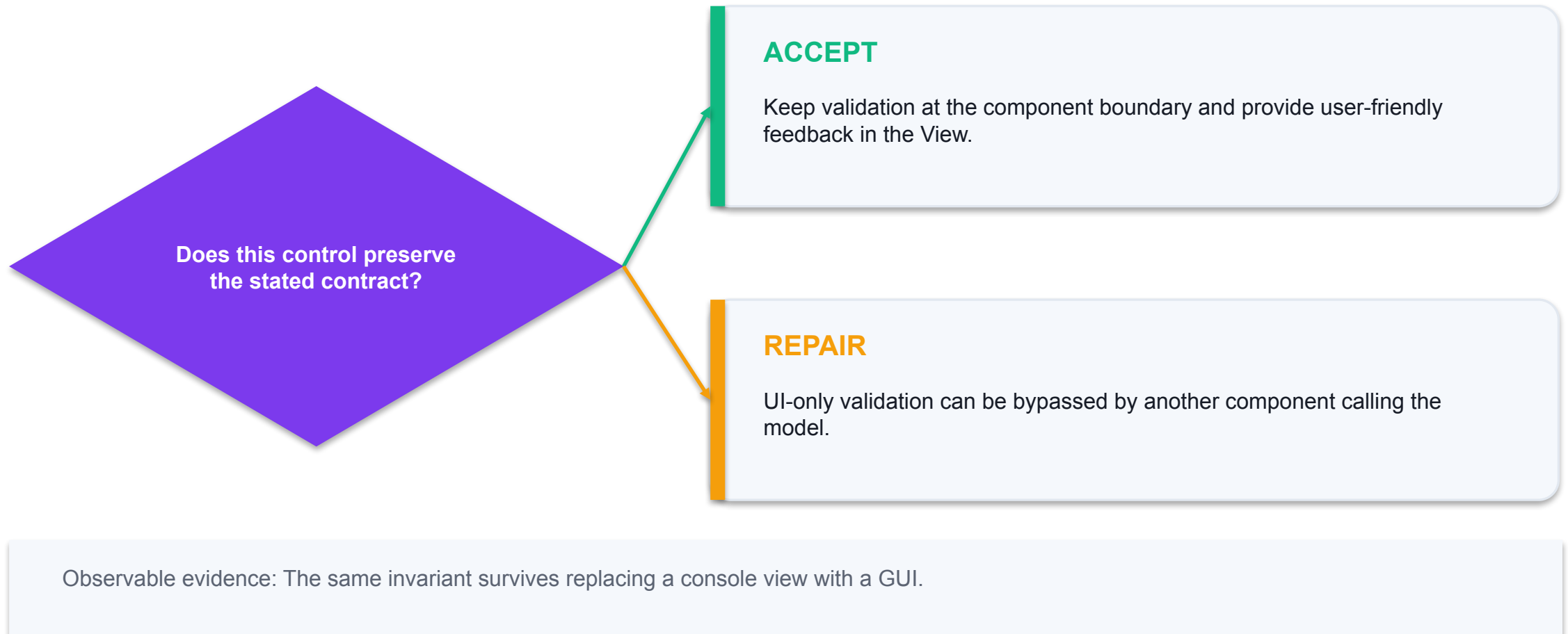
DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K1 / A1; apply to Activities 1–2.

Separate business rules from user interface controls — design decision



An AI prompt is a bounded design contract

Topic 1 · concrete C# mechanism and observable result

```
// Specification: R1 name required; R2 price > 0.  
// Target: C#, .NET 10, no new dependencies.  
// Output: design, minimal diff, boundary tests.  
// Evidence: actual compiler/run output only.
```

OBSERVATION

Constraints let the reviewer compare the proposed code against the same specification.

INPUT / TYPE

// Specification: R1 name required; R2 price > 0.

DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K1 / A1; apply to Activities 1–2.

An AI prompt is a bounded design contract — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	Constraints let the reviewer compare the proposed code against the same specification.	Check the stated input/type and observed result.
Failure path	An unbounded prompt may introduce packages, destructive operations or invented APIs.	Reproduce the smallest input that exposes this defect.
Design control	Approve a concrete diff only after explaining and testing it.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Review an AI patch against a failing example

Topic 1 · concrete C# mechanism and observable result

```
bool invalid = price < 0; // candidate bug
bool repaired = price <= 0; // correct R2
guard
// price=0: invalid False, repaired True.
```

OBSERVATION

One boundary input proves why the proposed operator must change.

INPUT / TYPE

bool invalid = price < 0; // candidate bug

DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K1 / A1; apply to Activities 1–2.

Review an AI patch against a failing example — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	One boundary input proves why the proposed operator must change.	Check the stated input/type and observed result.
Failure path	An AI assertion that code is correct is not execution evidence.	Reproduce the smallest input that exposes this defect.
Design control	Record input, expected/actual output, inspected diff and rerun result.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Compile-time, runtime and business failures differ

Topic 1 · concrete C# mechanism and observable result

```
// CS0103: misspelled identifier -> compile failure.  
// FormatException: invalid text -> runtime failure.  
// price=0 accepted -> business rule failure.
```

OBSERVATION

Different evidence points to a different repair layer.

INPUT / TYPE

// CS0103: misspelled identifier -> compile failure.

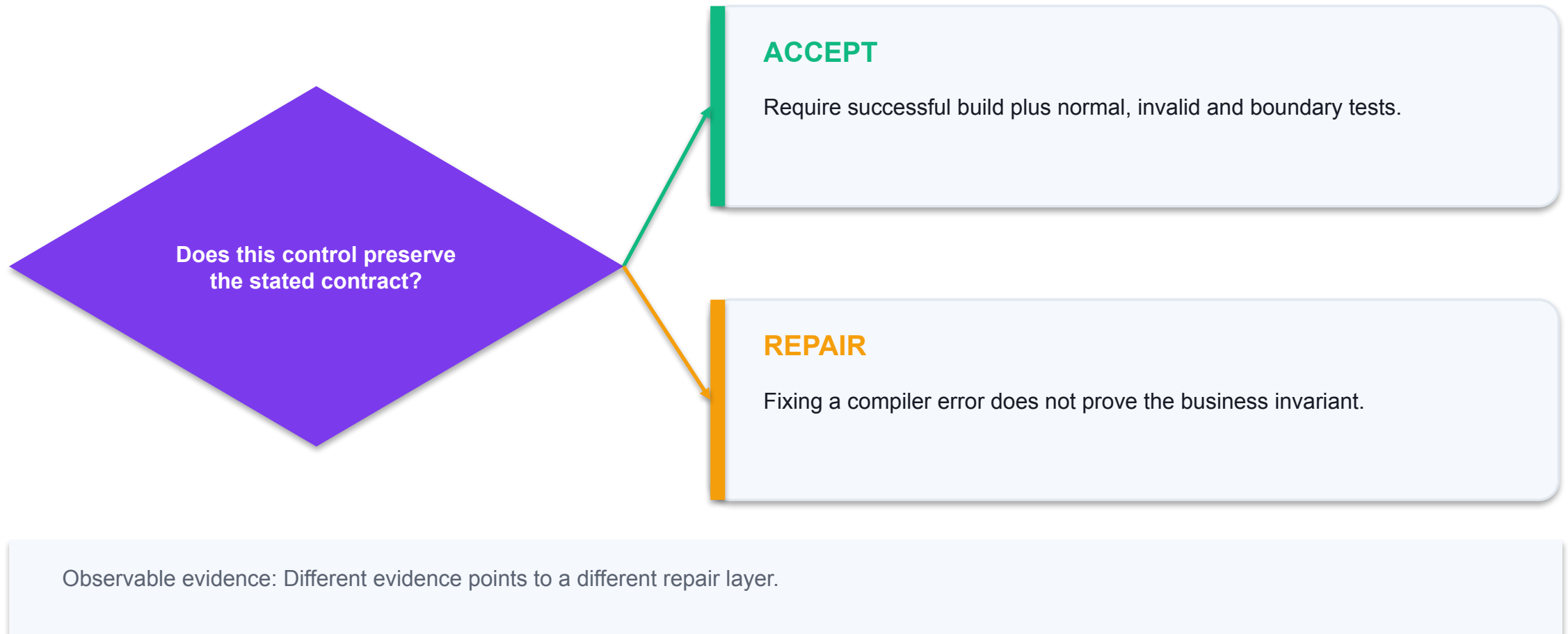
DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K1 / A1; apply to Activities 1–2.

Compile-time, runtime and business failures differ — design decision



C# and C++ share syntax, not runtime contracts

Topic 1 · concrete C# mechanism and observable result

```
// C#: List<int> quantities = new() {10,20};  
// C++ vector<int> is a comparison concept  
only.  
// C# uses using/IDisposable for owned  
resources.
```

OBSERVATION

The activities compile as C# projects and use the .NET library.

INPUT / TYPE

```
// C#: List<int> quantities = new() {10,20};
```

DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K1 / A1; apply to Activities 1–2.

C# and C++ share syntax, not runtime contracts — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	The activities compile as C# projects and use the .NET library.	Check the stated input/type and observed result.
Failure path	Pasting #include, std::cout or native pointer ownership into C# fails its language/runtime contract.	Reproduce the smallest input that exposes this defect.
Design control	Use the C++ links only to compare ideas; verify every C# API with Microsoft documentation.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

UML names a concrete component boundary

Topic 1 · concrete C# mechanism and observable result

```
// ProductCatalogue owns List<Product>.  
// Product validates its constructor inputs.  
// CatalogueView formats confirmation.  
// Add() mutates the catalogue after  
validation.
```

OBSERVATION

A class diagram shows ownership, visibility and relationships before code is expanded.

INPUT / TYPE

// ProductCatalogue owns List<Product>.

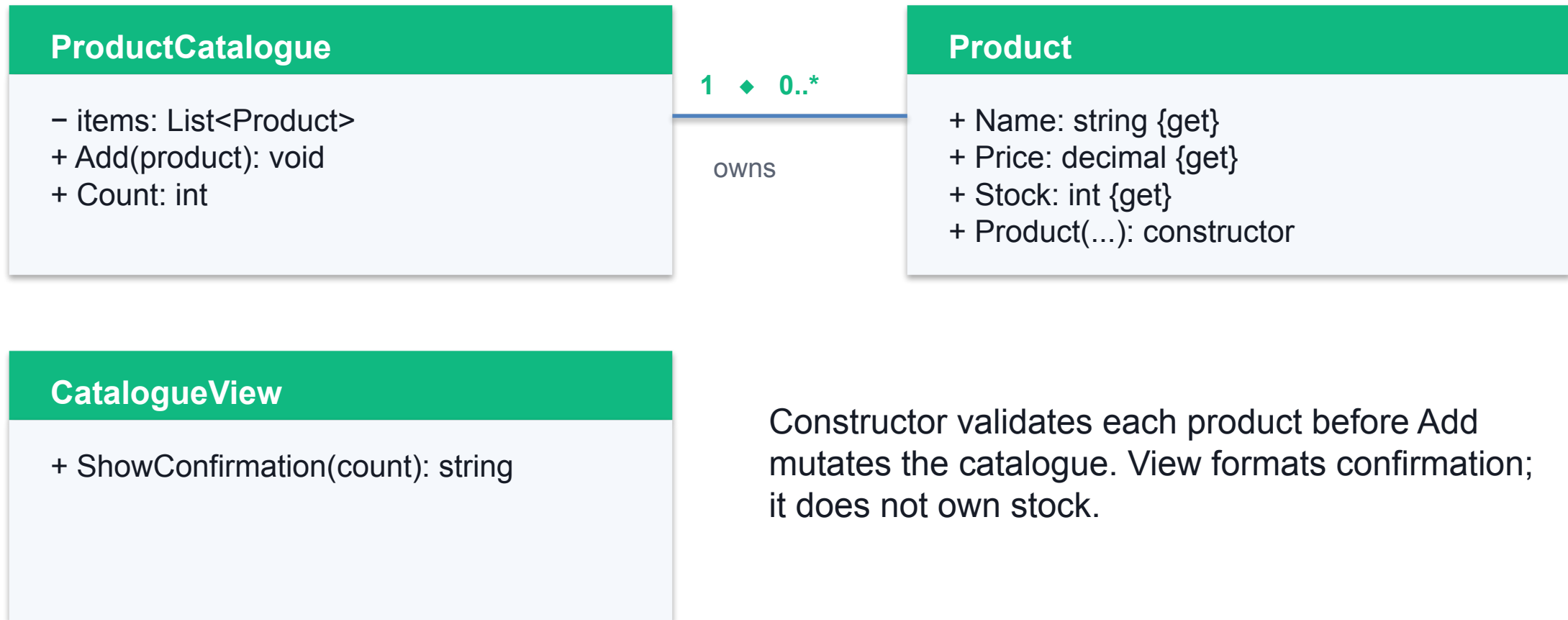
DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K1 / A1; apply to Activities 1–2.

Catalogue component — class diagram



Pseudocode and flowchart expose rejection paths

Topic 1 · concrete C# mechanism and observable result

```
// IF name blank: reject R1
// ELSE IF price <= 0: reject R2
// ELSE IF stock < 0: reject R3
// ELSE add and display confirmation
```

OBSERVATION

Ordering the guards makes the first rejected rule and success path clear.

INPUT / TYPE

// IF name blank: reject R1

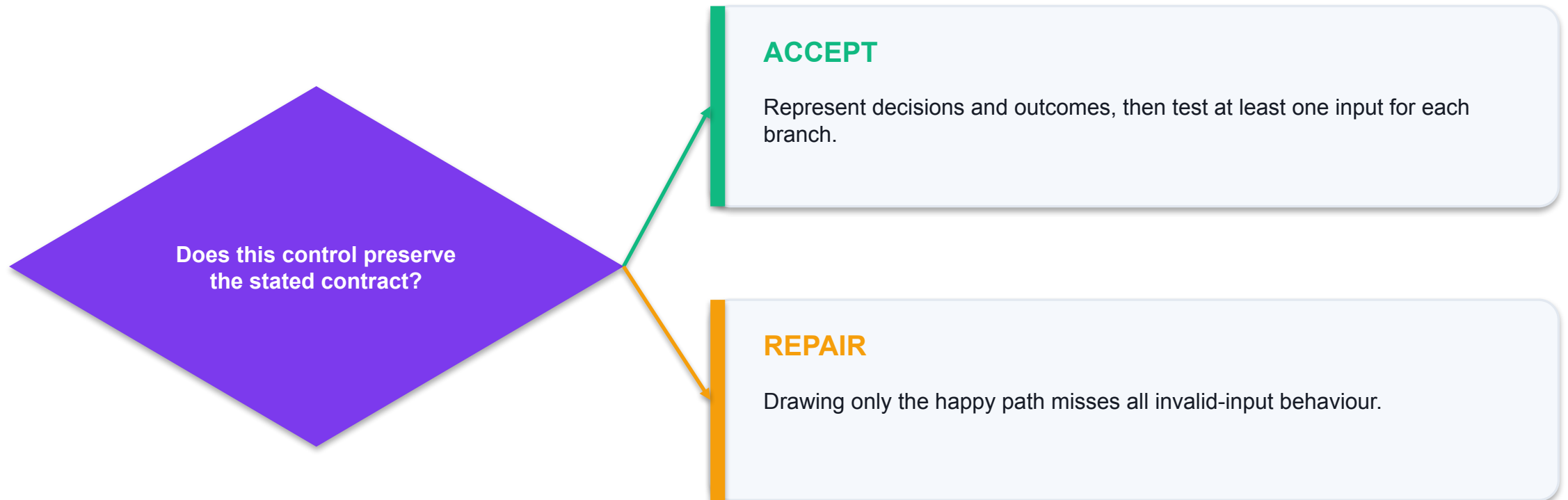
DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K1 / A1; apply to Activities 1–2.

Pseudocode and flowchart expose rejection paths — design decision



Observable evidence: Ordering the guards makes the first rejected rule and success path clear.

From catalogue specification to executable contract

ACTIVITY 01

LO1 · K1 · A1

A store needs a catalogue for Electronics, Clothing and Food. Reject blank names, prices at or below zero, negative stock and unknown categories. Define each requirement before generating a component.

YOU'LL BUILD

Runnable C# console component,
recorded tests and a reviewed design
decision

TOOLCHAIN

.NET 10 SDK · C# · local synthetic data

DONE WHEN

R1: blank names fail without creating a
record.

Activity 01 — vibe-coding prompt

Act as a C# reviewer. Work only in this .NET 10 console project. Read the synthetic specification: A store needs a catalogue for Electronics, Clothing and Food. Reject blank names, prices at or below zero, negative stock and unknown categories. Define each requirement before generating a component. Identify the input/output contract, invariants and error path. Return a short design and three test cases before proposing code. No new dependencies, network calls or file deletion.

INPUT

Synthetic specification + current source.

CONSTRAINT

C# / .NET 10; no new dependencies.

REVIEW

Minimal diff + concrete test cases.

EVIDENCE

Human explanation + actual execution.

Activity 01 — review and evidence prompts

REVIEW PROMPT

Review my Program.cs against requirements.md. Explain each defect using a concrete input and expected versus actual behaviour. Return the smallest C# diff and tests. Do not invent successful test results. If uncertain about an API, point to current Microsoft documentation.

EVIDENCE PROMPT

Using only my supplied actual build/run output, help me summarize one requirement-to-code-to-test link. Label any assumption. I must explain and approve the change myself.

Activity 01 — executable evidence

The baseline is supplied in this activity folder; full procedures are in the LG.

```
if (string.IsNullOrEmpty(name))  
    return "name";  
if (price <= 0)  
    return "price";  
if (stock < 0)  
    return "stock";
```

EXPECTED OUTPUT

Valid catalogue record: True
Invalid price -> price

INVARIANT

R1: blank names fail without creating a record.

FAILURE CHECK

R2: price must be strictly positive; stock zero is valid.

REVIEW TARGET

Write a requirements table with R1–R3, one example and one boundary case each.

Activity 01 — acceptance evidence

✓ Expected result

R1: blank names fail without creating a record.
R2: price must be strictly positive; stock zero is valid.
R3: only three named categories are accepted.
Self-test reports PASS: 5 contract checks.

IF IT DOESN'T WORK

Compiler failure

Inspect the first C# error and the project target.

Wrong result

Compare the actual input with the named business invariant.

AI mismatch

Reject an unverified API or out-of-scope dependency change.

Compile, run and review an AI-generated C# component

ACTIVITY 02

LO1 · K1 · A1

A greeter formats a warehouse welcome message. AI-generated text is a candidate change; compile it, test null/blank input and explain the result before accepting it.

YOU'LL BUILD

Runnable C# console component,
recorded tests and a reviewed design
decision

TOOLCHAIN

.NET 10 SDK · C# · local synthetic data

DONE WHEN

Whitespace and null input produce the
learner fallback.

Activity 02 — vibe-coding prompt

Act as a C# reviewer. Work only in this .NET 10 console project. Read the synthetic specification: A greeter formats a warehouse welcome message. AI-generated text is a candidate change; compile it, test null/blank input and explain the result before accepting it. Identify the input/output contract, invariants and error path. Return a short design and three test cases before proposing code. No new dependencies, network calls or file deletion.

INPUT

Synthetic specification + current source.

CONSTRAINT

C# / .NET 10; no new dependencies.

REVIEW

Minimal diff + concrete test cases.

EVIDENCE

Human explanation + actual execution.

Activity 02 — review and evidence prompts

REVIEW PROMPT

Review my Program.cs against requirements.md. Explain each defect using a concrete input and expected versus actual behaviour. Return the smallest C# diff and tests. Do not invent successful test results. If uncertain about an API, point to current Microsoft documentation.

EVIDENCE PROMPT

Using only my supplied actual build/run output, help me summarize one requirement-to-code-to-test link. Label any assumption. I must explain and approve the change myself.

Activity 02 — executable evidence

The baseline is supplied in this activity folder; full procedures are in the LG.

```
Console.WriteLine(Greet("Ada"));
Console.WriteLine(Greet(null));
// Greet checks absence before trimming name.
// Expected: Welcome, Ada!
// Expected: Welcome, learner!
```

EXPECTED OUTPUT

Welcome, Ada!
Welcome, learner!

INVARIANT

Whitespace and null input produce the learner fallback.

FAILURE CHECK

A non-empty name is trimmed before formatting.

REVIEW TARGET

Ask AI for an explanation of top-level statements, using only this file.

Activity 02 — acceptance evidence

✓ Expected result

Whitespace and null input produce the learner fallback.
A non-empty name is trimmed before formatting.
Program has one entry point and compiles without warnings.
Self-test reports PASS: 3 greeting checks.

IF IT DOESN'T WORK

Compiler failure

Inspect the first C# error and the project target.

Wrong result

Compare the actual input with the named business invariant.

AI mismatch

Reject an unverified API or out-of-scope dependency change.



TOPIC 2 · K2 / A2

C# programming fundamentals

Apply C# methodologies and tools for software creation.

Value types copy the value

Topic 2 · concrete C# mechanism and observable result

```
int stock = 10;  
int copy = stock;  
copy = 20;  
Console.WriteLine(stock); // 10
```

OBSERVATION

Changing the copy leaves the original integer at 10.

INPUT / TYPE

int stock = 10;

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

Value types copy the value — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	Changing the copy leaves the original integer at 10.	Check the stated input/type and observed result.
Failure path	Assuming two value variables share one storage location gives incorrect updates.	Reproduce the smallest input that exposes this defect.
Design control	Use int for bounded counts, decimal for currency and bool for conditions.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

References can share a mutable object

Topic 2 · concrete C# mechanism and observable result

```
var a = new List<int> { 10 };  
var b = a;  
b.Add(20);  
Console.WriteLine(a.Count); // 2
```

OBSERVATION

Both variables reference the same list; mutation is visible through either.

INPUT / TYPE

```
var a = new List<int> { 10 };
```

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

References can share a mutable object — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	Both variables reference the same list; mutation is visible through either.	Check the stated input/type and observed result.
Failure path	Returning a mutable internal list lets callers bypass validation.	Reproduce the smallest input that exposes this defect.
Design control	Expose a read-only view or deliberate copy at a component boundary.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Boxing and unboxing preserve runtime type

Topic 2 · concrete C# mechanism and observable result

```
object boxed = 10;  
int quantity = (int)boxed;  
// (long)boxed would fail: boxed value is an  
int.
```

OBSERVATION

A boxed int must be unboxed as int before a numeric conversion.

INPUT / TYPE

object boxed = 10;

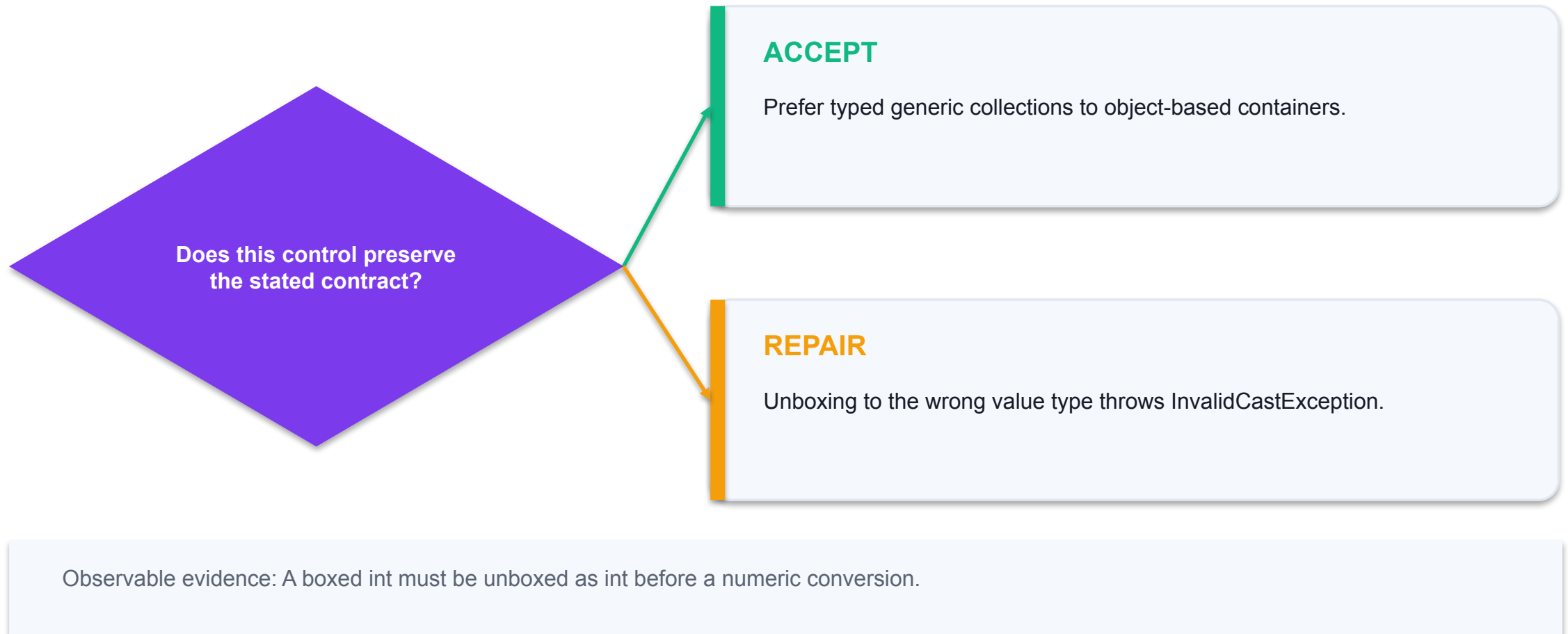
DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

Boxing and unboxing preserve runtime type — design decision



var and dynamic resolve at different times

Topic 2 · concrete C# mechanism and observable result

```
var quantity = 10; // compile-time int  
dynamic candidate = 10;  
Console.WriteLine(candidate + 2); // 12
```

OBSERVATION

var retains static checking; dynamic defers binding until execution.

INPUT / TYPE

var quantity = 10; // compile-time int

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

var and dynamic resolve at different times — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	var retains static checking; dynamic defers binding until execution.	Check the stated input/type and observed result.
Failure path	A nonexistent dynamic member fails at runtime instead of compile time.	Reproduce the smallest input that exposes this defect.
Design control	Keep domain components statically typed; confine dynamic to explicit interoperability boundaries.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Strings are immutable reference values

Topic 2 · concrete C# mechanism and observable result

```
string original = "Rice";  
string upper = original.ToUpperInvariant();  
Console.WriteLine(original); // Rice  
Console.WriteLine(upper); // RICE
```

OBSERVATION

Formatting returns a new string and leaves the original unchanged.

INPUT / TYPE

string original = "Rice";

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

Strings are immutable reference values — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	Formatting returns a new string and leaves the original unchanged.	Check the stated input/type and observed result.
Failure path	Treating string mutation as in-place mutation loses the returned result.	Reproduce the smallest input that exposes this defect.
Design control	Use invariant casing for identifiers and explicit culture for user-facing text.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Unsafe pointers are an advanced boundary

Topic 2 · concrete C# mechanism and observable result

```
// Concept excerpt; requires
AllowUnsafeBlocks=true.
unsafe {
    int value = 10; int* address = &value;
    Console.WriteLine(*address);
}
```

OBSERVATION

Pointer dereferencing reads the pointed-to integer.

INPUT / TYPE

// Concept excerpt; requires AllowUnsafeBlocks=true.

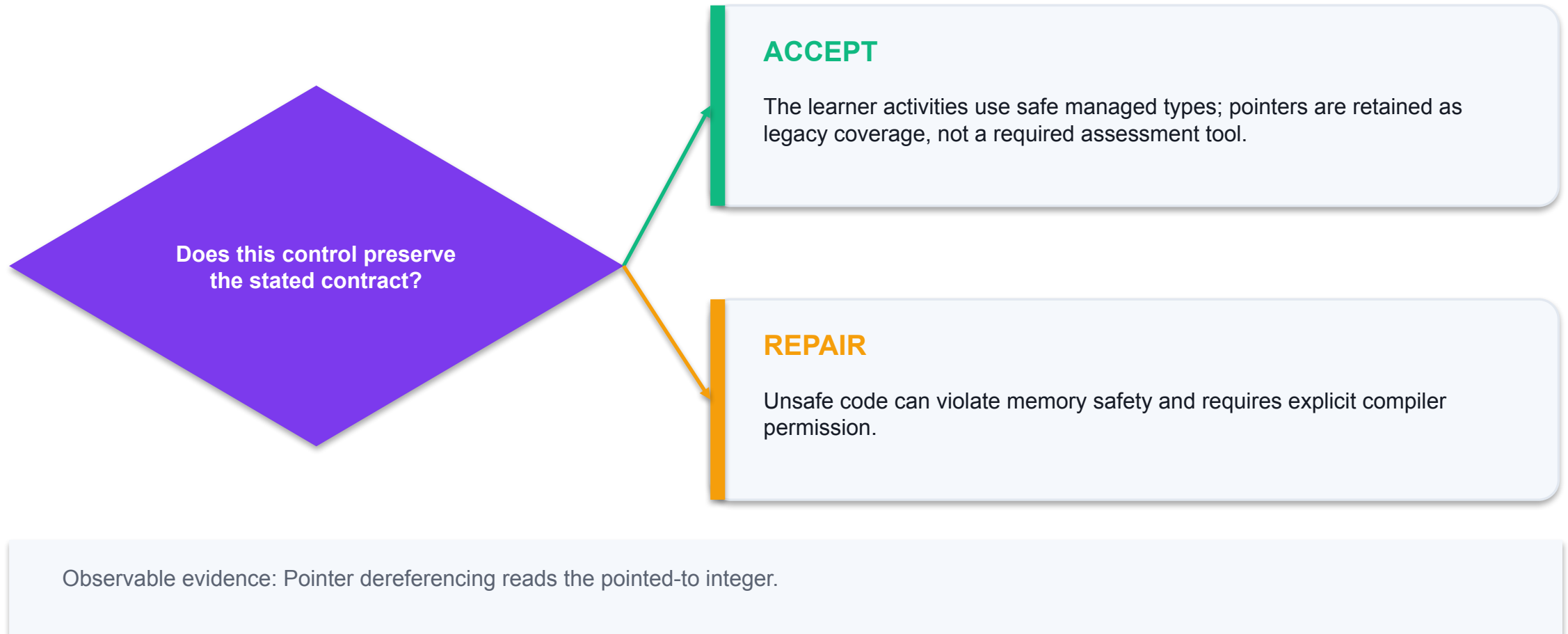
DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

Unsafe pointers are an advanced boundary — design decision



Implicit numeric conversion can lose precision

Topic 2 · concrete C# mechanism and observable result

```
int quantity = 10;  
long wide = quantity;  
int large = 16_777_217;  
float approximate = large;  
Console.WriteLine((int)approximate ==  
large); // False
```

OBSERVATION

Widening an integer range differs from preserving every significant digit.

INPUT / TYPE

int quantity = 10;

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

Implicit numeric conversion can lose precision — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	Widening an integer range differs from preserving every significant digit.	Check the stated input/type and observed result.
Failure path	int to float can round large exact integers.	Reproduce the smallest input that exposes this defect.
Design control	Choose types using the value range and precision contract, not only storage size.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Casts and conversion methods have distinct contracts

Topic 2 · concrete C# mechanism and observable result

```
double value = 2.9;
int truncated = (int)value; // 2
int rounded = Convert.ToInt32(value); // 3
Console.WriteLine($"{truncated}, {rounded}");
```

OBSERVATION

An explicit cast truncates this positive value; Convert rounds it.

INPUT / TYPE

double value = 2.9;

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

Casts and conversion methods have distinct contracts — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	An explicit cast truncates this positive value; Convert rounds it.	Check the stated input/type and observed result.
Failure path	Using the wrong conversion silently changes a business calculation.	Reproduce the smallest input that exposes this defect.
Design control	Document the rounding rule and test midpoint and negative values.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Definite assignment protects reads

Topic 2 · concrete C# mechanism and observable result

```
int quantity;  
quantity = 10;  
Console.WriteLine(quantity);  
// Reading before assignment is a compiler  
error.
```

OBSERVATION

A local must be assigned on every path before its value is read.

INPUT / TYPE

int quantity;

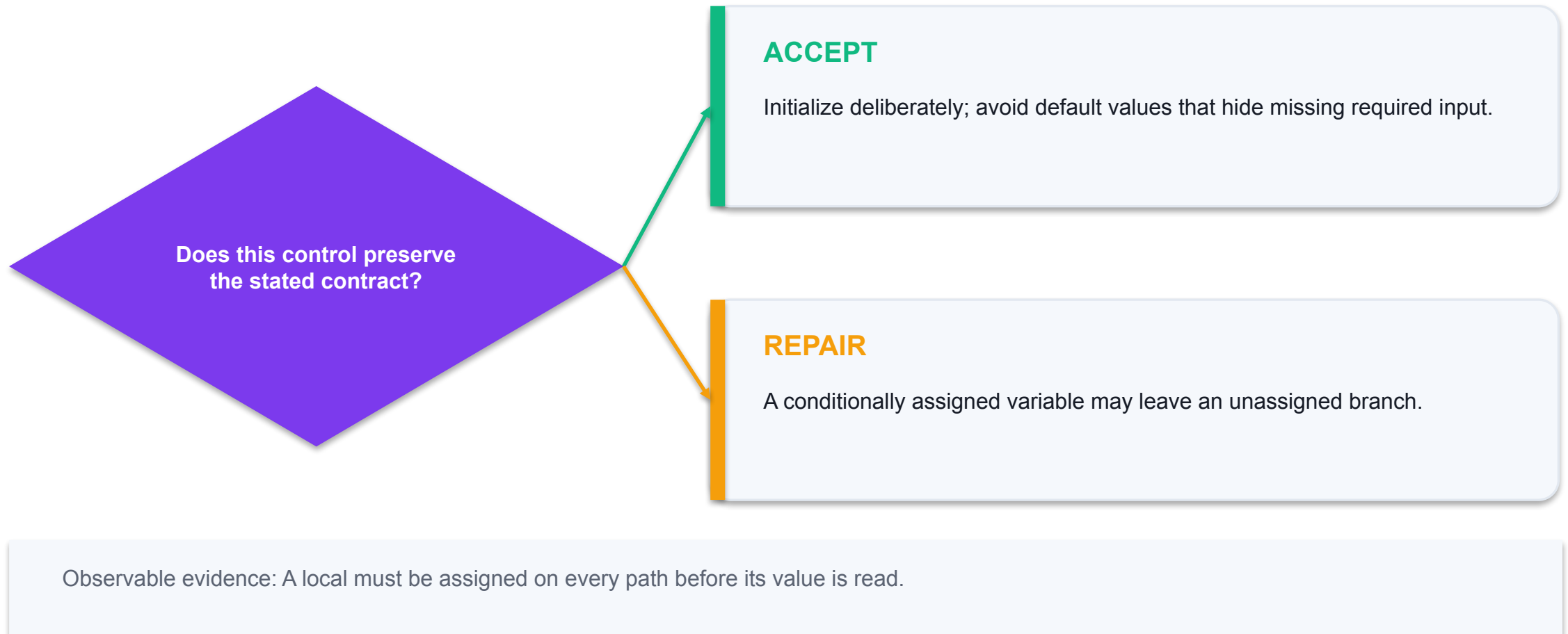
DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

Definite assignment protects reads — design decision



External text enters through a parsing guard

Topic 2 · concrete C# mechanism and observable result

```
string input = "two";  
bool parsed = int.TryParse(input, out int  
quantity);  
Console.WriteLine(parsed); // False  
// Do not use quantity unless parsed is true.
```

OBSERVATION

TryParse reports malformed input without throwing for normal bad text.

INPUT / TYPE

string input = "two";

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

External text enters through a parsing guard — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	TryParse reports malformed input without throwing for normal bad text.	Check the stated input/type and observed result.
Failure path	Parse on untrusted input can interrupt the workflow with FormatException.	Reproduce the smallest input that exposes this defect.
Design control	Validate syntax, range and domain invariant as separate controls.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Assignment target and evaluated expression

Topic 2 · concrete C# mechanism and observable result

```
int quantity = 10;  
quantity = quantity + 2;  
// quantity is assignable; quantity+2 is a  
value.  
Console.WriteLine(quantity); // 12
```

OBSERVATION

The right side is evaluated before it is stored in the target.

INPUT / TYPE

int quantity = 10;

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

Assignment target and evaluated expression — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	The right side is evaluated before it is stored in the target.	Check the stated input/type and observed result.
Failure path	A calculation expression cannot be assigned to as if it were a variable.	Reproduce the smallest input that exposes this defect.
Design control	Use property setters/methods where assignments must preserve invariants.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Literal suffixes define numeric intent

Topic 2 · concrete C# mechanism and observable result

```
int stock = 10;  
long large = 10_000_000_000L;  
decimal price = 19.90m;  
double ratio = 0.1;
```

OBSERVATION

The m suffix makes the price decimal rather than double.

INPUT / TYPE

int stock = 10;

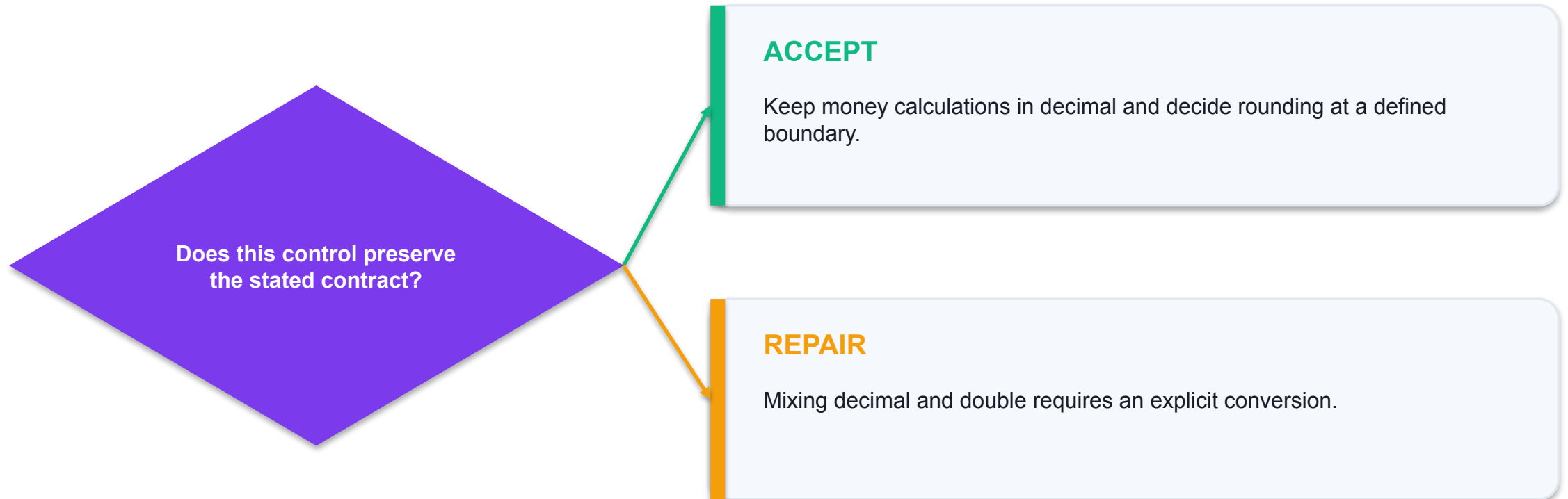
DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

Literal suffixes define numeric intent — design decision



Observable evidence: The m suffix makes the price decimal rather than double.

Characters, escaping and Unicode

Topic 2 · concrete C# mechanism and observable result

```
char newline = '\n';  
string path = @"C:\data\stock.json";  
string label = "米";  
Console.WriteLine(label);
```

OBSERVATION

Escapes represent control characters; Unicode text survives managed string handling.

INPUT / TYPE

char newline = '\n';

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

Characters, escaping and Unicode — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	Escapes represent control characters; Unicode text survives managed string handling.	Check the stated input/type and observed result.
Failure path	A char is one UTF-16 code unit and need not be a complete visible character.	Reproduce the smallest input that exposes this defect.
Design control	Use strings for names; preserve UTF-8 encoding in exported text files.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Constants and readonly values

Topic 2 · concrete C# mechanism and observable result

```
const decimal TaxRate = 0.09m;  
decimal price = 10m;  
decimal tax = price * TaxRate;  
Console.WriteLine(tax); // 0.90
```

OBSERVATION

const expresses a compile-time fixed value.

INPUT / TYPE

const decimal TaxRate = 0.09m;

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

Constants and readonly values — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	const expresses a compile-time fixed value.	Check the stated input/type and observed result.
Failure path	A const cannot depend on a value loaded at runtime.	Reproduce the smallest input that exposes this defect.
Design control	Use readonly for constructor-initialized state; keep time-sensitive business rates configurable.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Arithmetic and integer division

Topic 2 · concrete C# mechanism and observable result

```
Console.WriteLine(5 / 2); // 2  
Console.WriteLine(5m / 2); // 2.5  
Console.WriteLine(5 % 2); // 1
```

OBSERVATION

Operand types determine division behaviour; remainder has a separate contract.

INPUT / TYPE

Console.WriteLine(5 / 2); // 2

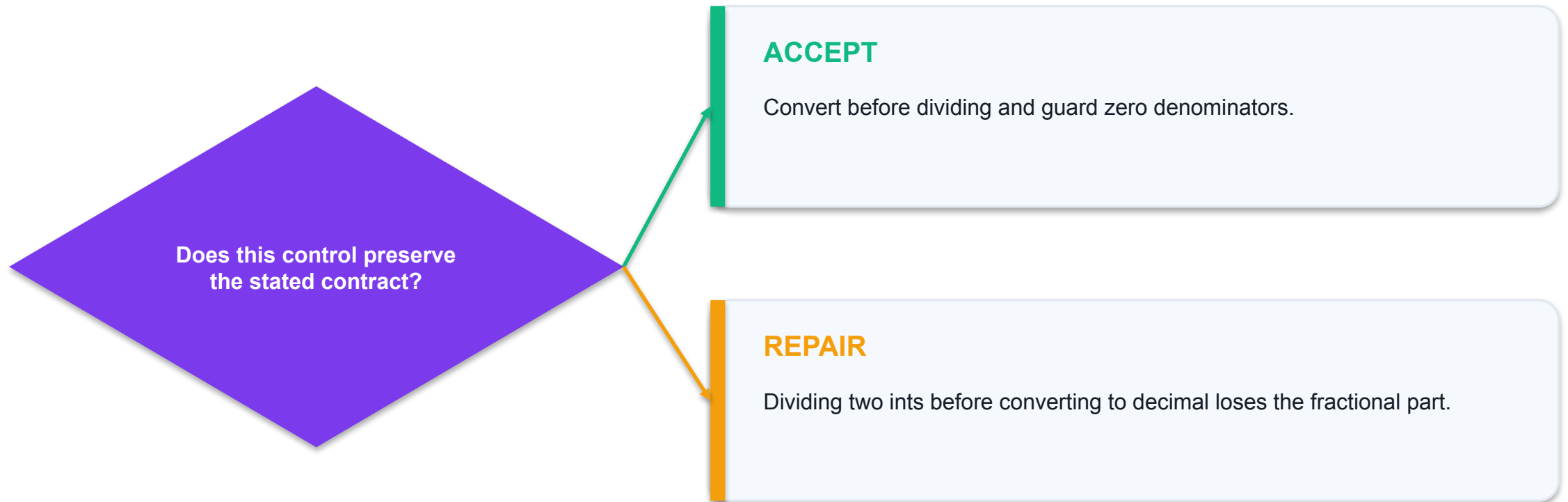
DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

Arithmetic and integer division — design decision



Observable evidence: Operand types determine division behaviour; remainder has a separate contract.

Relational operators encode boundaries

Topic 2 · concrete C# mechanism and observable result

```
int quantity = 0;  
Console.WriteLine(quantity >= 0); // True  
decimal price = 0m;  
Console.WriteLine(price > 0); // False
```

OBSERVATION

Zero stock is valid; zero price is invalid in the catalogue specification.

INPUT / TYPE

int quantity = 0;

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

Relational operators encode boundaries — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	Zero stock is valid; zero price is invalid in the catalogue specification.	Check the stated input/type and observed result.
Failure path	Using <code>>=</code> instead of <code>></code> for price admits a forbidden boundary.	Reproduce the smallest input that exposes this defect.
Design control	Translate words such as strictly positive into exact comparison operators.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Short-circuit logic prevents invalid reads

Topic 2 · concrete C# mechanism and observable result

```
string? name = null;  
bool valid = name is not null && name.Length >  
0;  
Console.WriteLine(valid); // False
```

OBSERVATION

&& skips the second operand when the first is false.

INPUT / TYPE

string? name = null;

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

Short-circuit logic prevents invalid reads — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	&& skips the second operand when the first is false.	Check the stated input/type and observed result.
Failure path	Using & evaluates both operands and may dereference null.	Reproduce the smallest input that exposes this defect.
Design control	Put the safety guard before the dependent access.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Bitwise flags are different from booleans

Topic 2 · concrete C# mechanism and observable result

```
[Flags] enum Permission { None=0, Read=1,
Write=2 }
// Within a method:
// var p = Permission.Read | Permission.Write;
// (p & Permission.Write) != 0 is true.
```

OBSERVATION

Flag values are independent bits combined with OR.

INPUT / TYPE

[Flags] enum Permission { None=0, Read=1, Write=2 }

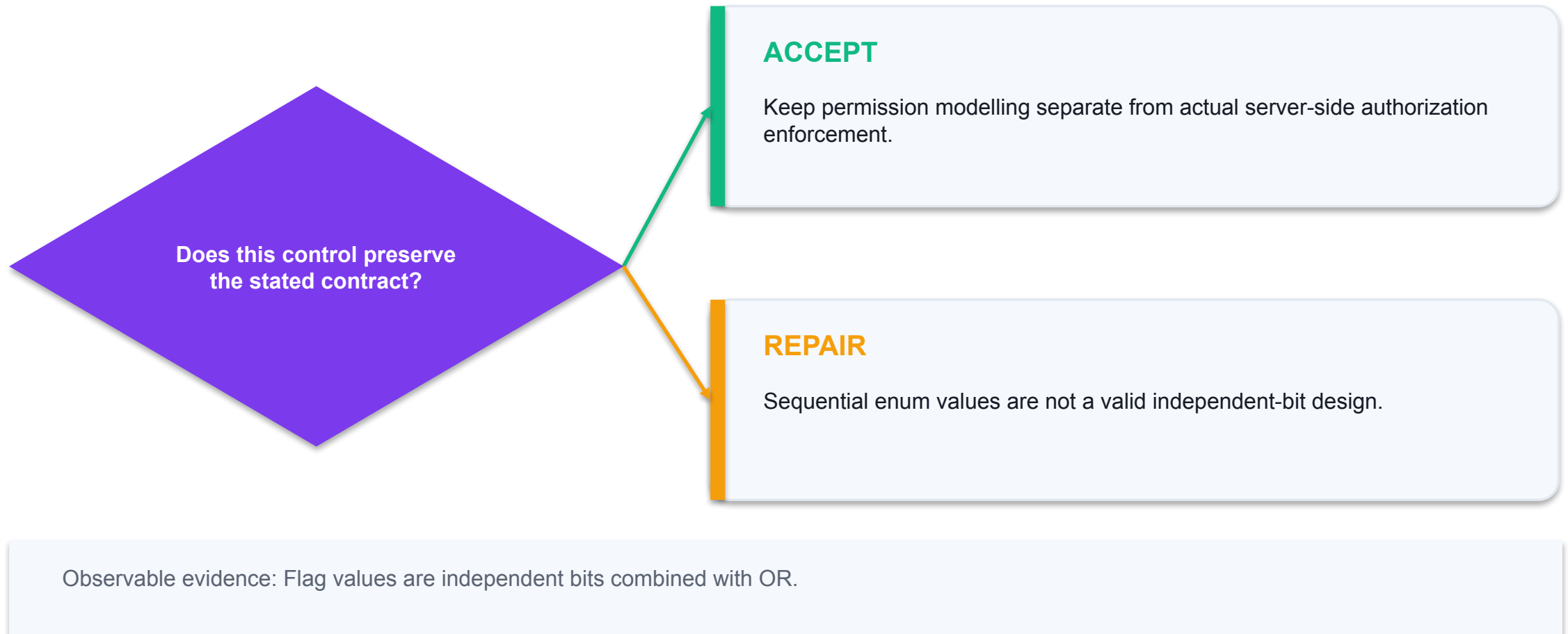
DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

Bitwise flags are different from booleans — design decision



Assignment, increment and checked arithmetic

Topic 2 · concrete C# mechanism and observable result

```
int quantity = 10;  
quantity += 2;  
int before = quantity++; // 12  
Console.WriteLine(quantity); // 13  
int safe = checked(quantity + 1);
```

OBSERVATION

Post-increment returns the old value; checked catches overflow.

INPUT / TYPE

int quantity = 10;

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

Assignment, increment and checked arithmetic — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	Post-increment returns the old value; checked catches overflow.	Check the stated input/type and observed result.
Failure path	An unchecked count overflow can produce a negative value.	Reproduce the smallest input that exposes this defect.
Design control	Avoid complex increment expressions; validate before committing a stock mutation.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Null and conditional operators

Topic 2 · concrete C# mechanism and observable result

```
string? supplied = null;  
string name = supplied ?? "learner";  
int stock = 0;  
string state = stock > 0 ? "available" :  
"empty";
```

OBSERVATION

?? chooses a null fallback; ?: chooses one expression from a boolean condition.

INPUT / TYPE

string? supplied = null;

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

Null and conditional operators — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	?? chooses a null fallback; ?: chooses one expression from a boolean condition.	Check the stated input/type and observed result.
Failure path	A fallback for null does not treat blank whitespace as missing.	Reproduce the smallest input that exposes this defect.
Design control	Use IsNullOrWhiteSpace when the domain contract rejects empty names.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Operator precedence changes the calculation

Topic 2 · concrete C# mechanism and observable result

```
decimal a = 10m + 2m * 3m; // 16  
decimal b = (10m + 2m) * 3m; // 36  
Console.WriteLine($"{a}, {b}");
```

OBSERVATION

Multiplication binds more tightly than addition.

INPUT / TYPE

decimal a = 10m + 2m * 3m; // 16

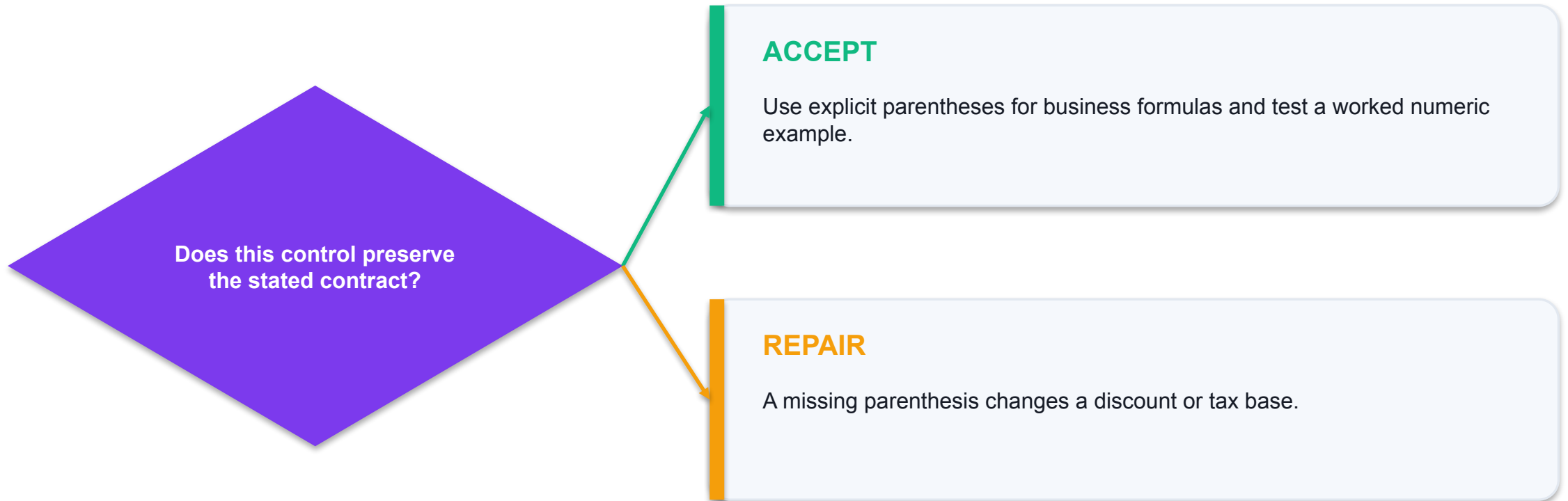
DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

Operator precedence changes the calculation — design decision



Observable evidence: Multiplication binds more tightly than addition.

if, else and switch choose one path

Topic 2 · concrete C# mechanism and observable result

```
int grade = 80;  
string band = grade switch {  
    >=90 => "A", >=80 => "B",  
    >=70 => "C", >=60 => "D", _ => "F"  
};
```

OBSERVATION

The first matching grade band handles 80 as B.

INPUT / TYPE

int grade = 80;

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

if, else and switch choose one path — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	The first matching grade band handles 80 as B.	Check the stated input/type and observed result.
Failure path	Placing a broad ≥ 60 case first hides higher bands.	Reproduce the smallest input that exposes this defect.
Design control	Order overlapping rules from specific/high thresholds to broader cases.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Loop termination is part of correctness

Topic 2 · concrete C# mechanism and observable result

```
int remaining = 3;
while (remaining > 0) {
    Console.WriteLine(remaining);
    remaining--;
}
```

OBSERVATION

The changing counter reaches the stopping condition.

INPUT / TYPE

int remaining = 3;

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

Loop termination is part of correctness — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	The changing counter reaches the stopping condition.	Check the stated input/type and observed result.
Failure path	A loop that never changes its condition becomes infinite.	Reproduce the smallest input that exposes this defect.
Design control	Bound retries and make the termination measure explicit.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

break and continue have different effects

Topic 2 · concrete C# mechanism and observable result

```
for (int i=0; i<5; i++) {  
    if (i==1) continue;  
    if (i==3) break;  
    Console.WriteLine(i);  
} // prints 0, 2
```

OBSERVATION

continue skips one iteration; break ends the nearest loop.

INPUT / TYPE

for (int i=0; i<5; i++) {

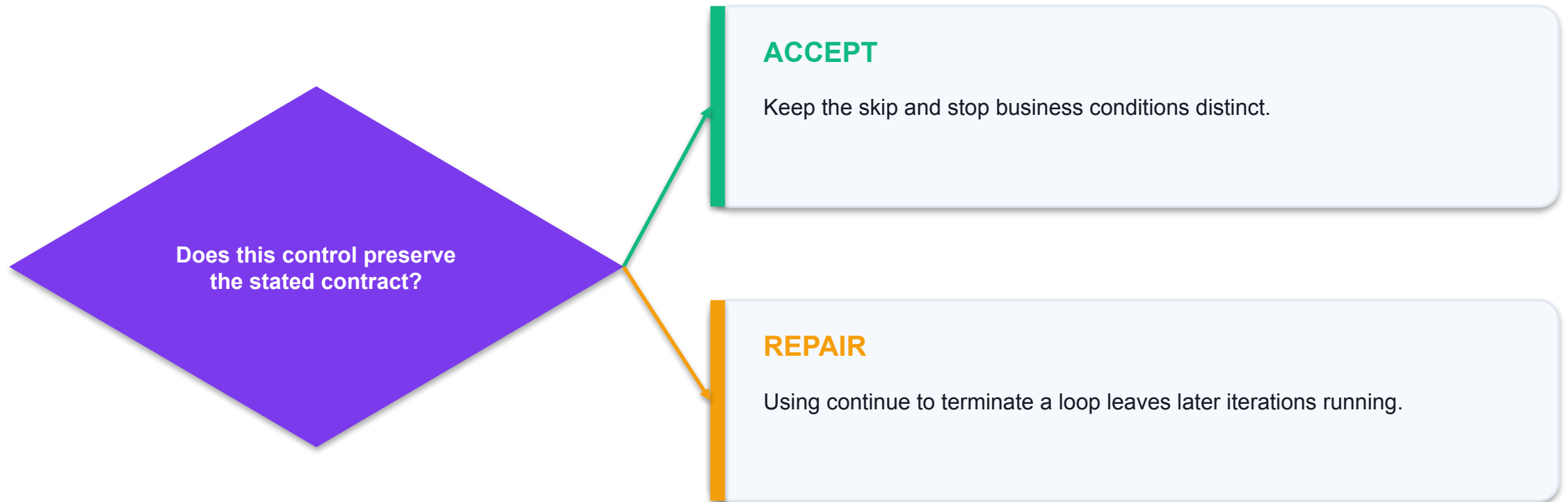
DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

break and continue have different effects — design decision



Observable evidence: continue skips one iteration; break ends the nearest loop.

Method inputs, outputs and responsibility

Topic 2 · concrete C# mechanism and observable result

```
static decimal Discount(decimal price, decimal
rate) {
    if (rate<0 || rate>1) throw new
ArgumentOutOfRangeException(nameof(rate));
    return price * (1-rate);
}
```

OBSERVATION

A pure method calculates a result from explicit inputs.

INPUT / TYPE

```
static decimal Discount(decimal price, decimal rate) {
```

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

Method inputs, outputs and responsibility — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	A pure method calculates a result from explicit inputs.	Check the stated input/type and observed result.
Failure path	Hidden global state makes tests depend on execution order.	Reproduce the smallest input that exposes this defect.
Design control	Name a method for one responsibility and specify valid inputs and return meaning.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Recursion needs a base case and input bound

Topic 2 · concrete C# mechanism and observable result

```
static int Factorial(int n) {  
    if(n<0 || n>12) throw new  
    ArgumentOutOfRangeException(nameof(n));  
    return n<=1 ? 1 : checked(n*Factorial(n-1));  
}
```

OBSERVATION

The base case returns 1; Factorial(5) returns 120.

INPUT / TYPE

```
static int Factorial(int n) {
```

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

Recursion needs a base case and input bound — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	The base case returns 1; Factorial(5) returns 120.	Check the stated input/type and observed result.
Failure path	Missing the base case causes unbounded recursion; large results overflow int.	Reproduce the smallest input that exposes this defect.
Design control	Prefer iteration when a simple counter expresses the same rule with less stack use.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Value, ref and out parameters

Topic 2 · concrete C# mechanism and observable result

```
static void Increment(ref int count) => count++;  
int quantity = 10;  
Increment(ref quantity); // quantity is 11  
bool ok = int.TryParse("12", out int parsed);
```

OBSERVATION

ref passes a writable storage location; out produces a value on each successful return path.

INPUT / TYPE

```
static void Increment(ref int count) => count++;
```

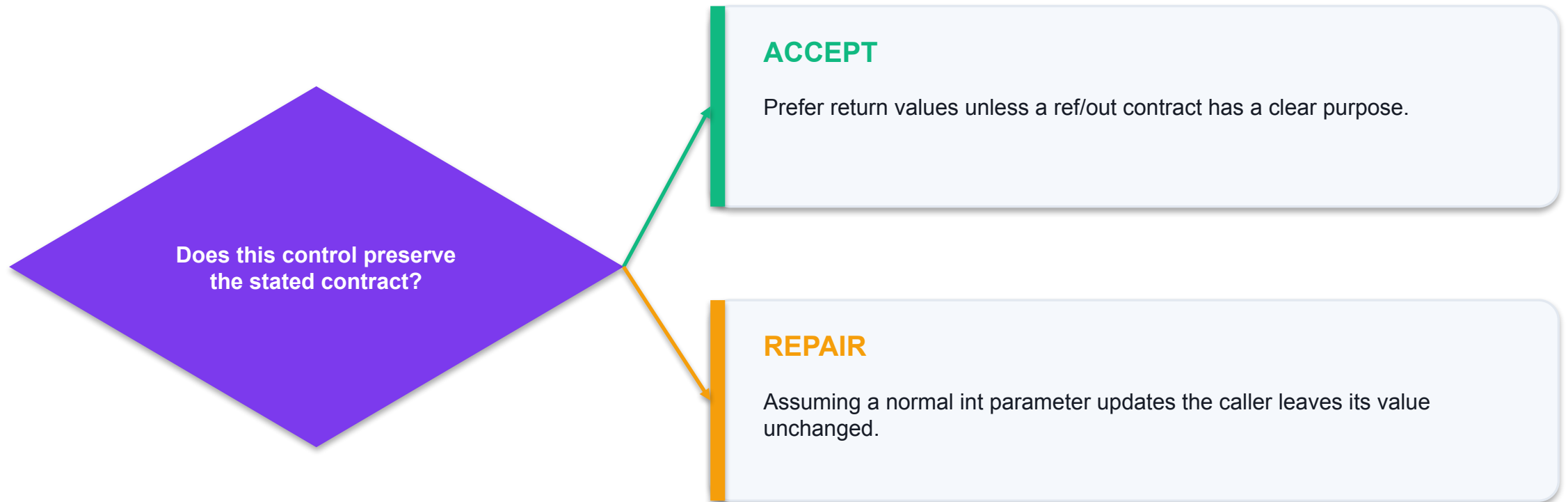
DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

Value, ref and out parameters — design decision



Observable evidence: ref passes a writable storage location; out produces a value on each successful return path.

Nullable values represent absence

Topic 2 · concrete C# mechanism and observable result

```
int? stock = null;  
Console.WriteLine(stock.HasValue); // False  
int displayStock = stock ?? 0;  
// null means unknown; 0 means known empty.
```

OBSERVATION

Nullable value types distinguish missing stock from zero stock.

INPUT / TYPE

int? stock = null;

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

Nullable values represent absence — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	Nullable value types distinguish missing stock from zero stock.	Check the stated input/type and observed result.
Failure path	Calling Value when HasValue is false throws.	Reproduce the smallest input that exposes this defect.
Design control	Do not collapse unknown into zero unless the display/use contract permits it.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Array rank and length are different

Topic 2 · concrete C# mechanism and observable result

```
int[,] grid = new int[2,3];  
Console.WriteLine(grid.Rank); // 2  
Console.WriteLine(grid.Length); // 6  
Console.WriteLine(grid.GetLength(0)); // 2
```

OBSERVATION

Rank counts dimensions; Length counts all elements.

INPUT / TYPE

```
int[,] grid = new int[2,3];
```

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

Array rank and length are different — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	Rank counts dimensions; Length counts all elements.	Check the stated input/type and observed result.
Failure path	Confusing rank with size breaks indexing and traversal.	Reproduce the smallest input that exposes this defect.
Design control	Use zero-based indexes and compare each dimension against its own length.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

foreach and array variants

Topic 2 · concrete C# mechanism and observable result

```
int[] grades = [90,80,70];  
int sum = 0;  
foreach(int grade in grades) sum += grade;  
decimal mean = (decimal)sum / grades.Length;
```

OBSERVATION

foreach visits each value; casting before division gives mean 80.

INPUT / TYPE

int[] grades = [90,80,70];

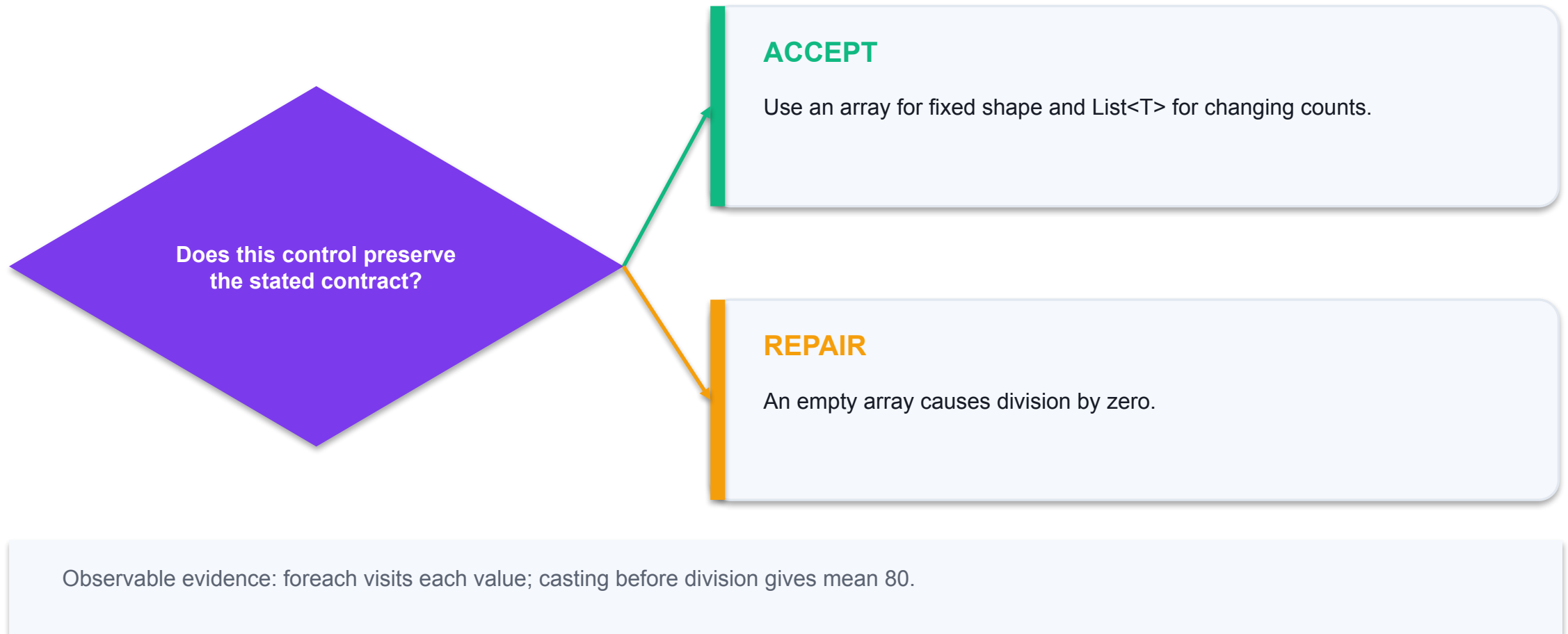
DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

foreach and array variants — design decision



Constructing and formatting strings

Topic 2 · concrete C# mechanism and observable result

```
string product = "Rice";  
decimal price = 2.50m;  
string line = $"{product}: {price:F2}";  
Console.WriteLine(line);
```

OBSERVATION

Interpolation combines values with explicit numeric formatting.

INPUT / TYPE

string product = "Rice";

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

Constructing and formatting strings — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	Interpolation combines values with explicit numeric formatting.	Check the stated input/type and observed result.
Failure path	Culture-dependent output can break machine-to-machine comparisons.	Reproduce the smallest input that exposes this defect.
Design control	Use explicit/invariant culture for exchange formats and the user's locale for display.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Ordinal comparison and search

Topic 2 · concrete C# mechanism and observable result

```
string title = "C# Design";  
bool match = title.Contains("c#",  
StringComparison.OrdinalIgnoreCase);  
Console.WriteLine(match); // True
```

OBSERVATION

The comparison contract is explicitly case-insensitive and ordinal.

INPUT / TYPE

string title = "C# Design";

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

Ordinal comparison and search — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	The comparison contract is explicitly case-insensitive and ordinal.	Check the stated input/type and observed result.
Failure path	Default culture-sensitive comparisons may surprise identifier matching.	Reproduce the smallest input that exposes this defect.
Design control	Select comparison mode according to human-language versus identifier semantics.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Substring, join and range guards

Topic 2 · concrete C# mechanism and observable result

```
string sku = "SKU-001";  
string suffix = sku.Substring(4,3);  
string joined = string.Join(", ", new[]  
{"Rice", "Tea"});
```

OBSERVATION

Substring selects a bounded range; Join inserts a delimiter between values.

INPUT / TYPE

string sku = "SKU-001";

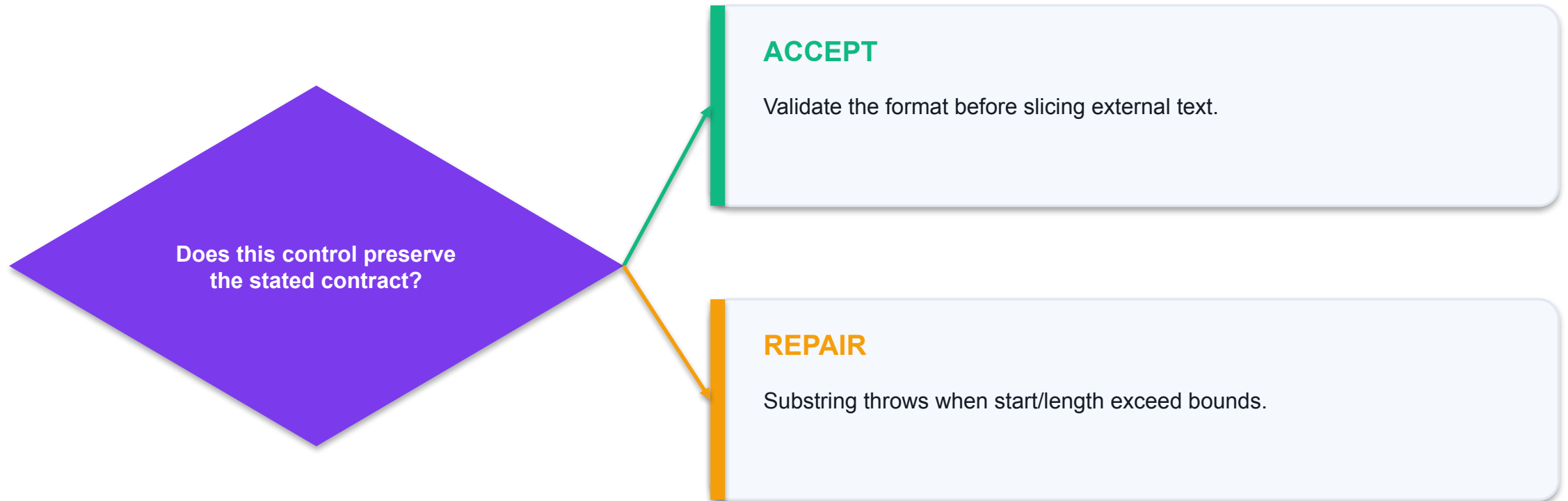
DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K2 / A2; apply to Activities 3–4.

Substring, join and range guards — design decision



Observable evidence: Substring selects a bounded range; Join inserts a delimiter between values.

MVC makes coordination explicit

Topic 2 · concrete C# mechanism and observable result

```
// Model: BankAccount.Balance  
// View: BalanceView.Format(number,balance)  
// Controller: BalanceController.Enquire()  
// Result: DEMO-001: balance 125.50
```

OBSERVATION

The Controller connects a domain value to its display without duplicating either responsibility.

INPUT / TYPE

// Model: BankAccount.Balance

DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K2 / A2; apply to Activities 3–4.

MVC makes coordination explicit — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	The Controller connects a domain value to its display without duplicating either responsibility.	Check the stated input/type and observed result.
Failure path	Putting balance validation in the View creates inconsistent business rules.	Reproduce the smallest input that exposes this defect.
Design control	Use the console MVC example as a small design technique before adopting a web framework.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Boundary tests target the exact grade thresholds

Topic 2 · concrete C# mechanism and observable result

```
// Band(89) -> B; Band(90) -> A  
// Band(79) -> C; Band(80) -> B  
// Band(59) -> F; Band(60) -> D
```

OBSERVATION

Adjacent inputs reveal off-by-one comparison errors.

INPUT / TYPE

// Band(89) -> B; Band(90) -> A

DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K2 / A2; apply to Activities 3–4.

Boundary tests target the exact grade thresholds — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	Adjacent inputs reveal off-by-one comparison errors.	Check the stated input/type and observed result.
Failure path	Testing only 95 never distinguishes >90 from >=90.	Reproduce the smallest input that exposes this defect.
Design control	For each threshold T, test T-1, T and T+1 where the input domain permits.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Money rounding belongs to a named boundary

Topic 2 · concrete C# mechanism and observable result

```
decimal total = 19.90m * 2 * (1-0.10m);  
decimal receipt =  
decimal.Round(total,2,MidpointRounding.AwayFromZero);  
// Receipt: 35.82
```

OBSERVATION

A documented rounding rule makes receipt output reproducible.

INPUT / TYPE

decimal total = 19.90m * 2 * (1-0.10m);

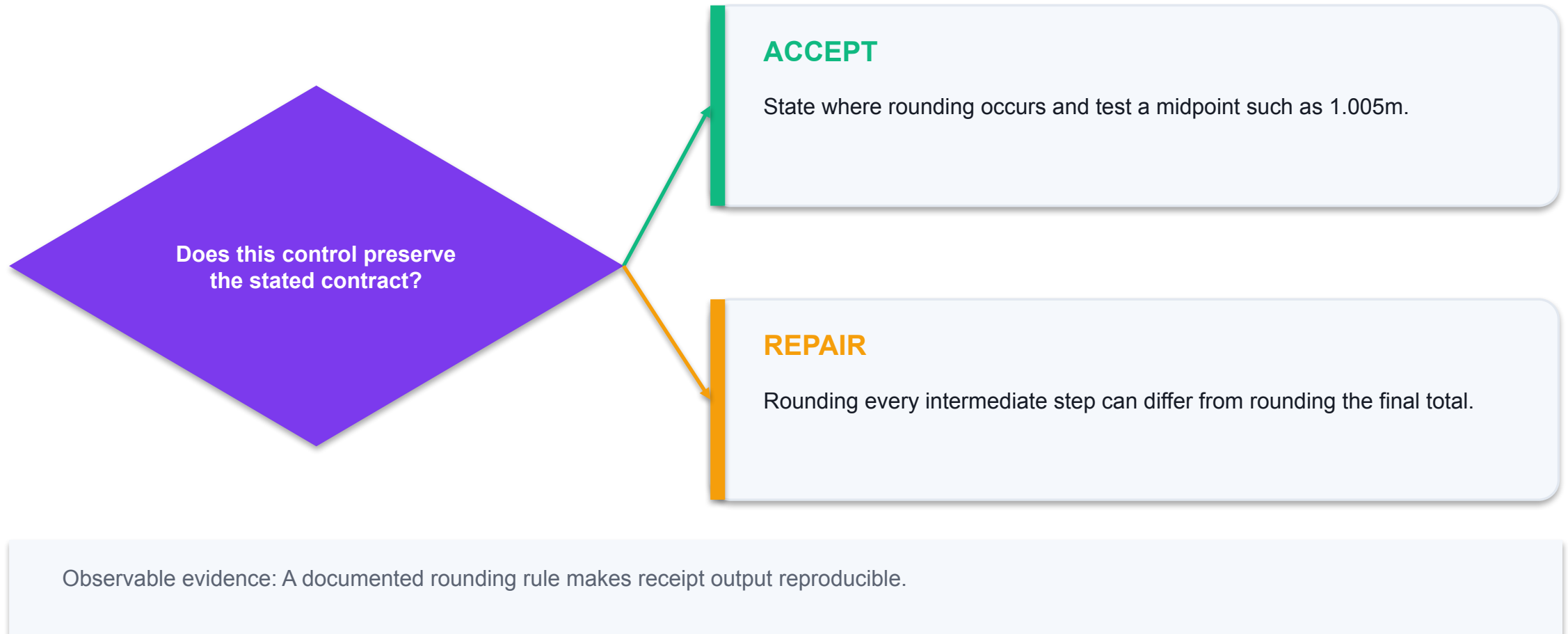
DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K2 / A2; apply to Activities 3–4.

Money rounding belongs to a named boundary — design decision



A nullable reference warning reveals missing handling

Topic 2 · concrete C# mechanism and observable result

```
string? input = null;  
string label =  
string.IsNullOrEmpty(input)  
    ? "learner" : input.Trim();
```

OBSERVATION

The guard establishes a usable non-empty label without disabling nullable analysis.

INPUT / TYPE

string? input = null;

DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K2 / A2; apply to Activities 3–4.

A nullable reference warning reveals missing handling — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	The guard establishes a usable non-empty label without disabling nullable analysis.	Check the stated input/type and observed result.
Failure path	Adding ! suppresses the warning but does not make a null value safe.	Reproduce the smallest input that exposes this defect.
Design control	Treat nullability annotations as an input contract; repair the path rather than suppressing it.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Collection shape determines the update contract

Topic 2 · concrete C# mechanism and observable result

```
int[] fixedGrades = [90,80,70];  
var growingProducts = new List<string>();  
growingProducts.Add("Rice");
```

OBSERVATION

An array has fixed length; a List grows through an explicit operation.

INPUT / TYPE

int[] fixedGrades = [90,80,70];

DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K2 / A2; apply to Activities 3–4.

Collection shape determines the update contract — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	An array has fixed length; a List grows through an explicit operation.	Check the stated input/type and observed result.
Failure path	Choosing an array then assuming Add exists breaks the API expectation.	Reproduce the smallest input that exposes this defect.
Design control	Match the container to fixed dimensions, growing collections and lookup requirements.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Types, operators, input rules and batch control flow

ACTIVITY 03

LO2 · K2 · A2

A receipt calculator prices two items and totals a grade batch. Currency uses decimal, quantities use int, discounts are bounded, and invalid external text is parsed safely.

YOU'LL BUILD

Runnable C# console component,
recorded tests and a reviewed design
decision

TOOLCHAIN

.NET 10 SDK · C# · local synthetic data

DONE WHEN

$19.90 \times 2 \times 0.90$ produces 35.82.

Activity 03 — vibe-coding prompt

Act as a C# reviewer. Work only in this .NET 10 console project. Read the synthetic specification: A receipt calculator prices two items and totals a grade batch. Currency uses decimal, quantities use int, discounts are bounded, and invalid external text is parsed safely. Identify the input/output contract, invariants and error path. Return a short design and three test cases before proposing code. No new dependencies, network calls or file deletion.

INPUT

Synthetic specification + current source.

CONSTRAINT

C# / .NET 10; no new dependencies.

REVIEW

Minimal diff + concrete test cases.

EVIDENCE

Human explanation + actual execution.

Activity 03 — review and evidence prompts

REVIEW PROMPT

Review my Program.cs against requirements.md. Explain each defect using a concrete input and expected versus actual behaviour. Return the smallest C# diff and tests. Do not invent successful test results. If uncertain about an API, point to current Microsoft documentation.

EVIDENCE PROMPT

Using only my supplied actual build/run output, help me summarize one requirement-to-code-to-test link. Label any assumption. I must explain and approve the change myself.

Activity 03 — executable evidence

The baseline is supplied in this activity folder; full procedures are in the LG.

```
Console.WriteLine(
    $"Receipt total: {Total(19.90m, 2,
0.10m):F2}");
int[] grades = [90, 80, 70];
int sum = 0;
foreach (int grade in grades) sum += grade;
// Total = 35.82; mean = 80; band = B.
```

EXPECTED OUTPUT

Receipt total: 35.82
Mean grade: 80.00
Grade band: B

INVARIANT

$19.90 \times 2 \times 0.90$ produces 35.82.

FAILURE CHECK

Malformed quantities are rejected with TryParse.

REVIEW TARGET

Add 59/60,69/70,79/80 and 89/90 grade boundary checks.

Activity 03 — acceptance evidence

✓ Expected result

19.90 × 2 × 0.90 produces 35.82.

Malformed quantities are rejected with TryParse.

Grade 90 is A and 89 is B.

Negative quantities fail; discount remains between 0 and 1.

IF IT DOESN'T WORK

Compiler failure

Inspect the first C# error and the project target.

Wrong result

Compare the actual input with the named business invariant.

AI mismatch

Reject an unverified API or out-of-scope dependency change.

Methods and console MVC for balance enquiries

ACTIVITY 04

LO2 · K2 · A2

A bank requires console MVC: Model owns a non-negative balance, View formats output, Controller coordinates. Public members use PascalCase and local/private variables camelCase.

YOU'LL BUILD

Runnable C# console component,
recorded tests and a reviewed design
decision

TOOLCHAIN

.NET 10 SDK · C# · local synthetic data

DONE WHEN

The Model rejects negative initial
balance.

Activity 04 — vibe-coding prompt

Act as a C# reviewer. Work only in this .NET 10 console project. Read the synthetic specification: A bank requires console MVC: Model owns a non-negative balance, View formats output, Controller coordinates. Public members use PascalCase and local/private variables camelCase. Identify the input/output contract, invariants and error path. Return a short design and three test cases before proposing code. No new dependencies, network calls or file deletion.

INPUT

Synthetic specification + current source.

CONSTRAINT

C# / .NET 10; no new dependencies.

REVIEW

Minimal diff + concrete test cases.

EVIDENCE

Human explanation + actual execution.

Activity 04 — review and evidence prompts

REVIEW PROMPT

Review my Program.cs against requirements.md. Explain each defect using a concrete input and expected versus actual behaviour. Return the smallest C# diff and tests. Do not invent successful test results. If uncertain about an API, point to current Microsoft documentation.

EVIDENCE PROMPT

Using only my supplied actual build/run output, help me summarize one requirement-to-code-to-test link. Label any assumption. I must explain and approve the change myself.

Activity 04 — executable evidence

The baseline is supplied in this activity folder; full procedures are in the LG.

```
var account = new BankAccount("DEMO-001",
    125.50m);
var view = new BalanceView();
var controller = new
    BalanceController(account, view);
Console.WriteLine(controller.Enquire());
// DEMO-001: balance 125.50
// Controller coordinates model and view.
```

EXPECTED OUTPUT

DEMO-001: balance 125.50
Factorial(5): 120

INVARIANT

The Model rejects negative initial balance.

FAILURE CHECK

View owns formatting; Controller holds no account rule.

REVIEW TARGET

Add nullable account lookup and a view error message without moving business rules into the View.

Activity 04 — acceptance evidence

✓ Expected result

The Model rejects negative initial balance.
View owns formatting; Controller holds no account rule.
Factorial(0) is 1 and Factorial(5) is 120.
Public method/property names follow PascalCase.

IF IT DOESN'T WORK

Compiler failure

Inspect the first C# error and the project target.

Wrong result

Compare the actual input with the named business invariant.

AI mismatch

Reject an unverified API or out-of-scope dependency change.

TOPIC 3 · K3 / A3

Classes, objects and design controls

Select essential C# controls and features to meet software design requirements.

Access modifiers define a visibility boundary

Topic 3 · concrete C# mechanism and observable result

```
public class Base {  
    protected internal int shared;  
    private protected int family;  
    private int secret = 10;  
    public int ReadSecret() => secret;  
}
```

OBSERVATION

protected internal allows same assembly OR derived type;
private protected requires same assembly and
containing/derived type.

INPUT / TYPE

public class Base {

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K3 / A3; apply to Activities 5–6.

Access modifiers define a visibility boundary — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	protected internal allows same assembly OR derived type; private protected requires same assembly and containing/derived type.	Check the stated input/type and observed result.
Failure path	Visibility is not identity verification or authorization.	Reproduce the smallest input that exposes this defect.
Design control	Expose the smallest necessary surface; a top-level class defaults internal and a nested class defaults private.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Classes model identity and controlled state

Topic 3 · concrete C# mechanism and observable result

```
sealed class Stock {  
    public int Count { get; private set; }  
    public Stock(int initial) {  
        if(initial<0) throw new  
ArgumentOutOfRangeException(nameof(initial));  
        Count=initial;  
    }  
}
```

OBSERVATION

Construction starts with a valid state and a private setter limits mutation paths.

INPUT / TYPE

sealed class Stock {

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K3 / A3; apply to Activities 5–6.

Classes model identity and controlled state — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	Construction starts with a valid state and a private setter limits mutation paths.	Check the stated input/type and observed result.
Failure path	A public unchecked setter can admit negative stock.	Reproduce the smallest input that exposes this defect.
Design control	Put invariants in the component; let UI controls collect and display values.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Constructors initialize complete objects

Topic 3 · concrete C# mechanism and observable result

```
sealed class Product {  
    public string Name { get; }  
    public Product() : this("Unspecified") { }  
    public Product(string name) {  
        if(string.IsNullOrEmpty(name)) throw  
new ArgumentException("Name required");  
        Name=name;  
    }  
}
```

OBSERVATION

Constructor chaining reuses one validation path.

INPUT / TYPE

sealed class Product {

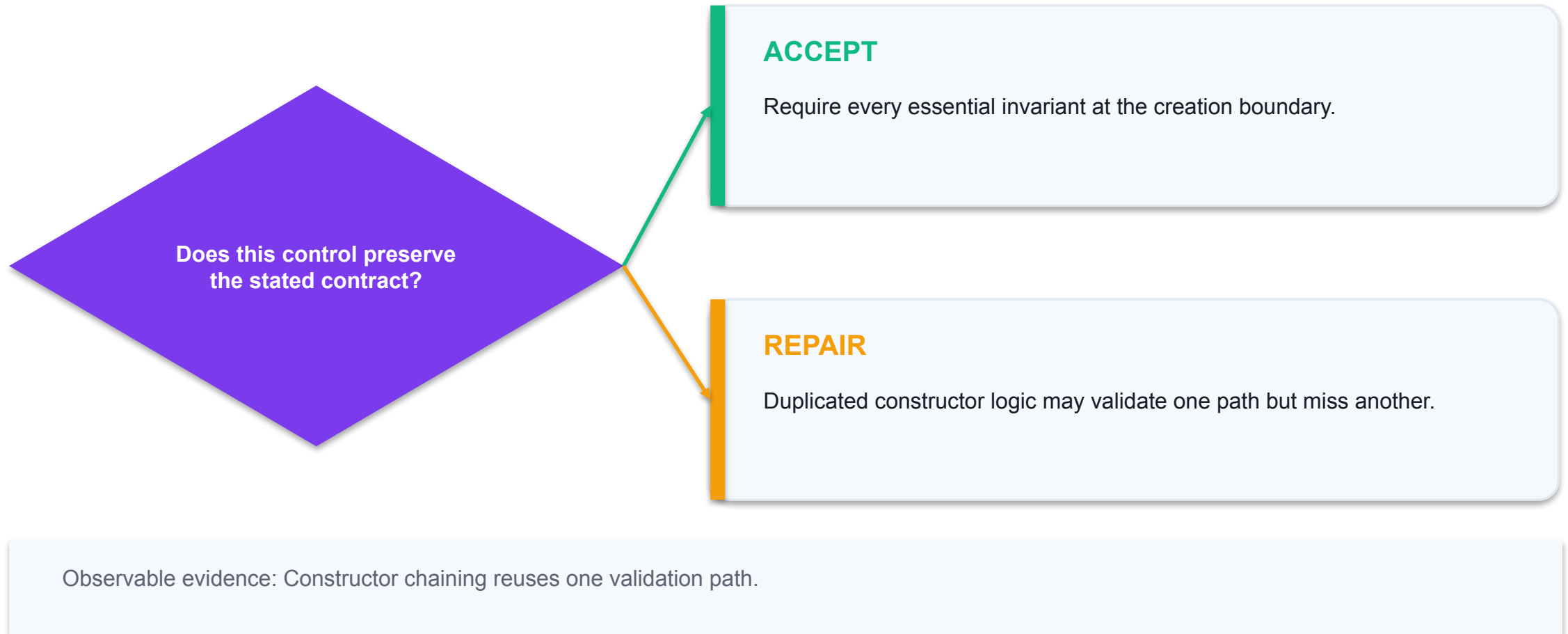
DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K3 / A3; apply to Activities 5–6.

Constructors initialize complete objects — design decision



Finalizers are not reliable cleanup

Topic 3 · concrete C# mechanism and observable result

```
// Deterministic disposal:  
using var writer = new  
StreamWriter("stock.txt");  
writer.WriteLine("SKU-001,10");  
//  
Type() is a nondeterministic finalizer.
```

OBSERVATION

using disposes a resource at the end of scope, including exceptional exits.

INPUT / TYPE

// Deterministic disposal:

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K3 / A3; apply to Activities 5–6.

Finalizers are not reliable cleanup — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	using disposes a resource at the end of scope, including exceptional exits.	Check the stated input/type and observed result.
Failure path	Waiting for a finalizer may leave a file handle open for an unknown time.	Reproduce the smallest input that exposes this defect.
Design control	Use IDisposable/using for resources; implement a finalizer only for justified unmanaged ownership.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Static state has process-level lifetime

Topic 3 · concrete C# mechanism and observable result

```
static class FinePolicy {  
    public const decimal DailyRate = 0.50m;  
    public static decimal Calculate(int  
        overdueDays)  
        => Math.Max(0,overdueDays) * DailyRate;  
}
```

OBSERVATION

A stateless policy is shared without per-object state.

INPUT / TYPE

```
static class FinePolicy {
```

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K3 / A3; apply to Activities 5–6.

Static state has process-level lifetime — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	A stateless policy is shared without per-object state.	Check the stated input/type and observed result.
Failure path	Mutable static counters persist across unrelated tests and callers.	Reproduce the smallest input that exposes this defect.
Design control	Prefer stateless static functions or explicit scoped service instances.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Inheritance and base construction

Topic 3 · concrete C# mechanism and observable result

```
abstract class Person(string name) {  
    public string Name { get; } = name;  
}  
sealed class Student(string name) :  
    Person(name) { }
```

OBSERVATION

A derived constructor supplies the base state.

INPUT / TYPE

```
abstract class Person(string name) {
```

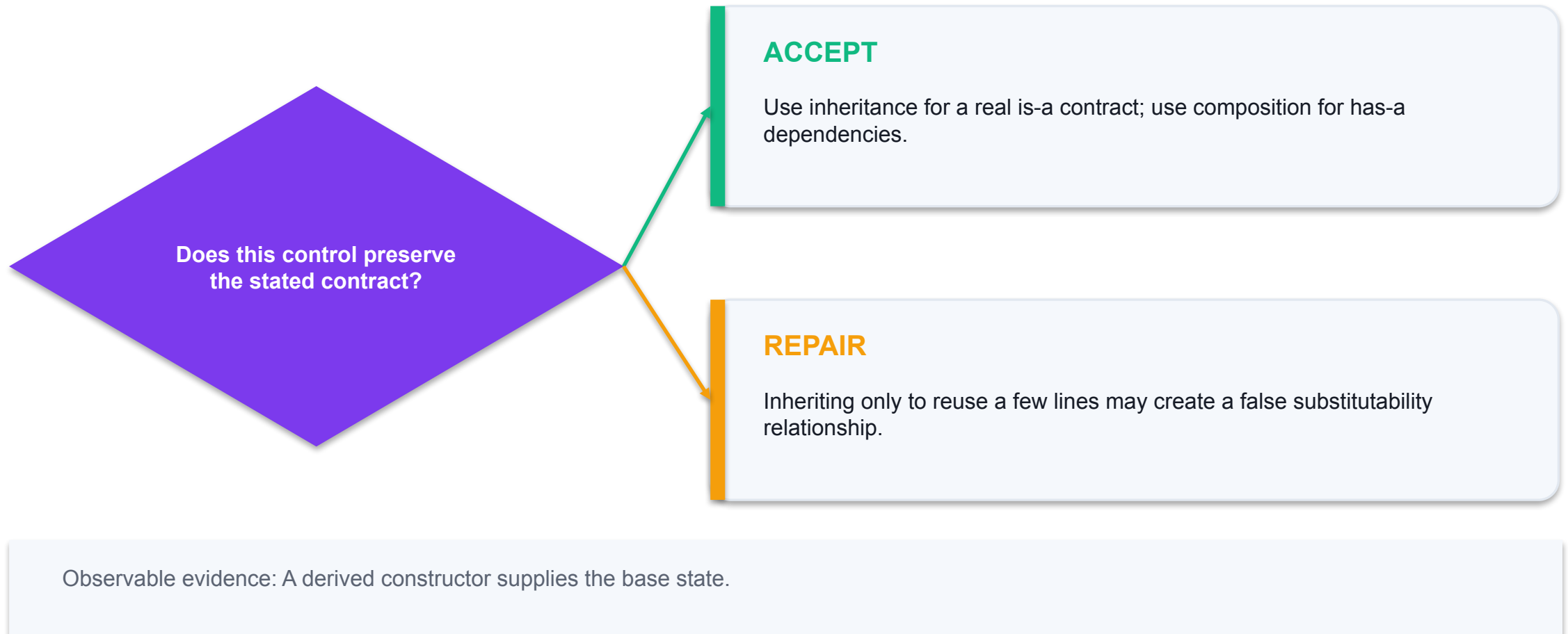
DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K3 / A3; apply to Activities 5–6.

Inheritance and base construction — design decision



One class base, multiple interfaces

Topic 3 · concrete C# mechanism and observable result

```
interface IReadable { string Read(); }
interface IWritable { void Write(string value); }
class Store : IReadable, IWritable {
    public string Read() => "demo";
    public void Write(string value) { }
}
```

OBSERVATION

A class may implement multiple contracts but inherit one class base.

INPUT / TYPE

```
interface IReadable { string Read(); }
```

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K3 / A3; apply to Activities 5–6.

One class base, multiple interfaces — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	A class may implement multiple contracts but inherit one class base.	Check the stated input/type and observed result.
Failure path	Two base classes are not supported in a C# class declaration.	Reproduce the smallest input that exposes this defect.
Design control	Use interfaces to decouple callers from interchangeable components.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Overloads select by compile-time signature

Topic 3 · concrete C# mechanism and observable result

```
static string Show(int quantity) => $"count {quantity}";  
static string Show(decimal price) => $"price {price:F2}";  
Console.WriteLine(Show(10));  
Console.WriteLine(Show(10m));
```

OBSERVATION

Argument types choose different overloads before execution.

INPUT / TYPE

static string Show(int quantity) => \$"count {quantity}";

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K3 / A3; apply to Activities 5–6.

Overloads select by compile-time signature — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	Argument types choose different overloads before execution.	Check the stated input/type and observed result.
Failure path	Two methods cannot differ only by return type.	Reproduce the smallest input that exposes this defect.
Design control	Keep overload meanings consistent and avoid ambiguous optional parameters.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Virtual dispatch chooses runtime behaviour

Topic 3 · concrete C# mechanism and observable result

```
abstract class Policy { public abstract string  
Band(decimal mean); }  
sealed class Standard : Policy {  
    public override string Band(decimal mean) =>  
mean >= 90 ? "A" : "B";  
}  
// Policy p = new Standard(); p.Band(85)  
returns B.
```

OBSERVATION

A base-typed reference dispatches to the derived override.

INPUT / TYPE

```
abstract class Policy { public abstract string Band(decimal  
mean); }
```

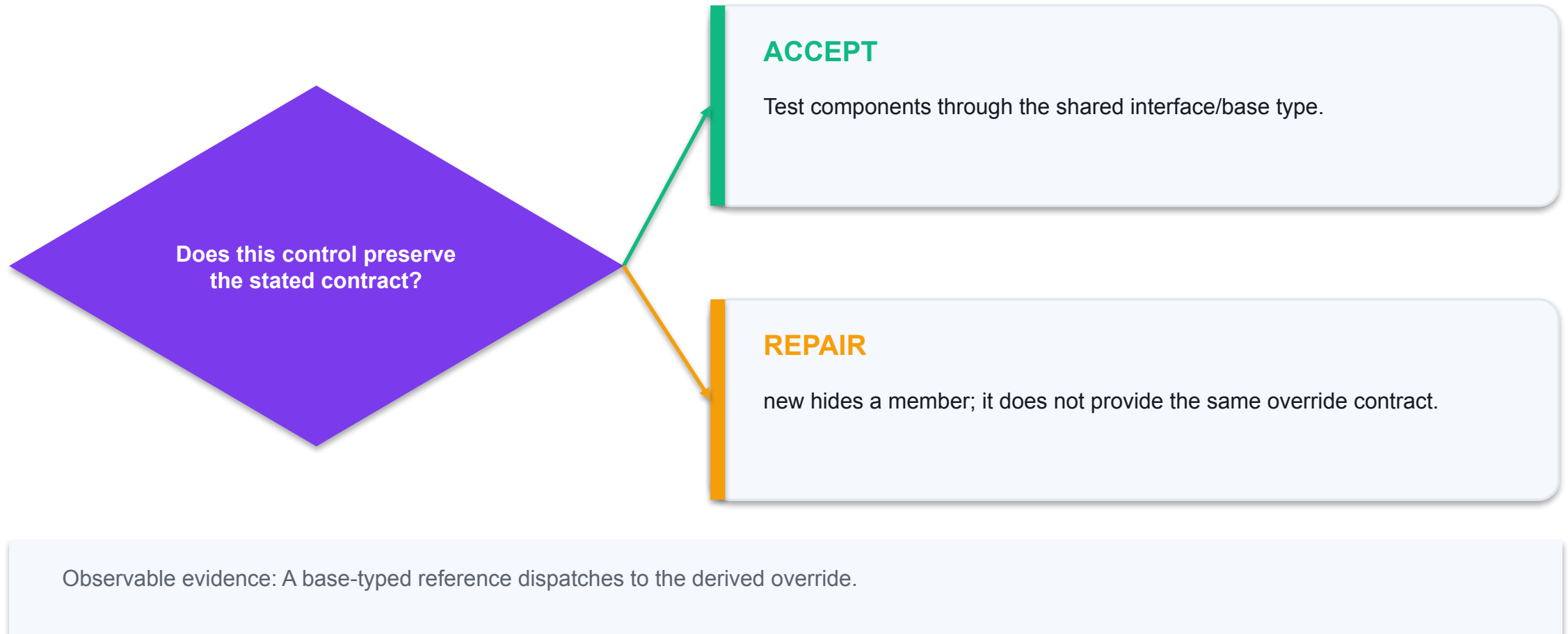
DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K3 / A3; apply to Activities 5–6.

Virtual dispatch chooses runtime behaviour — design decision



Operator overloads must preserve meaning

Topic 3 · concrete C# mechanism and observable result

```
readonly record struct Money(decimal Amount) {  
    public static Money operator +(Money a,  
    Money b)  
        => new(a.Amount+b.Amount);  
}  
// new Money(2m)+new Money(3m) -> Amount 5m
```

OBSERVATION

A user type can define meaningful addition while remaining immutable.

INPUT / TYPE

readonly record struct Money(decimal Amount) {

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K3 / A3; apply to Activities 5–6.

Operator overloads must preserve meaning — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	A user type can define meaningful addition while remaining immutable.	Check the stated input/type and observed result.
Failure path	Surprising operator semantics make callers misread the code.	Reproduce the smallest input that exposes this defect.
Design control	Prefer explicit named methods when the operation is not naturally an operator.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Properties express controlled mutation

Topic 3 · concrete C# mechanism and observable result

```
public int Stock { get; private set; }  
public void AdjustStock(int delta) {  
    int next = checked(Stock + delta);  
    if(next<0) throw new  
        InvalidOperationException("Insufficient  
        stock");  
    Stock=next;  
}
```

OBSERVATION

Validation computes the candidate before assigning, preserving the old state on rejection.

INPUT / TYPE

```
public int Stock { get; private set; }
```

DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K3 / A3; apply to Activities 5–6.

Properties express controlled mutation — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	Validation computes the candidate before assigning, preserving the old state on rejection.	Check the stated input/type and observed result.
Failure path	Subtracting first and validating afterwards leaves an invalid object.	Reproduce the smallest input that exposes this defect.
Design control	Make rejected changes atomic at the local component level.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Composition injects interchangeable services

Topic 3 · concrete C# mechanism and observable result

```
// OrderProcessor(PaymentService payment,  
//   NotificationService notification, string  
//   logPath)  
// calls payment before notification.
```

OBSERVATION

Dependencies are visible in construction and can be replaced by local mocks.

INPUT / TYPE

// OrderProcessor(PaymentService payment,

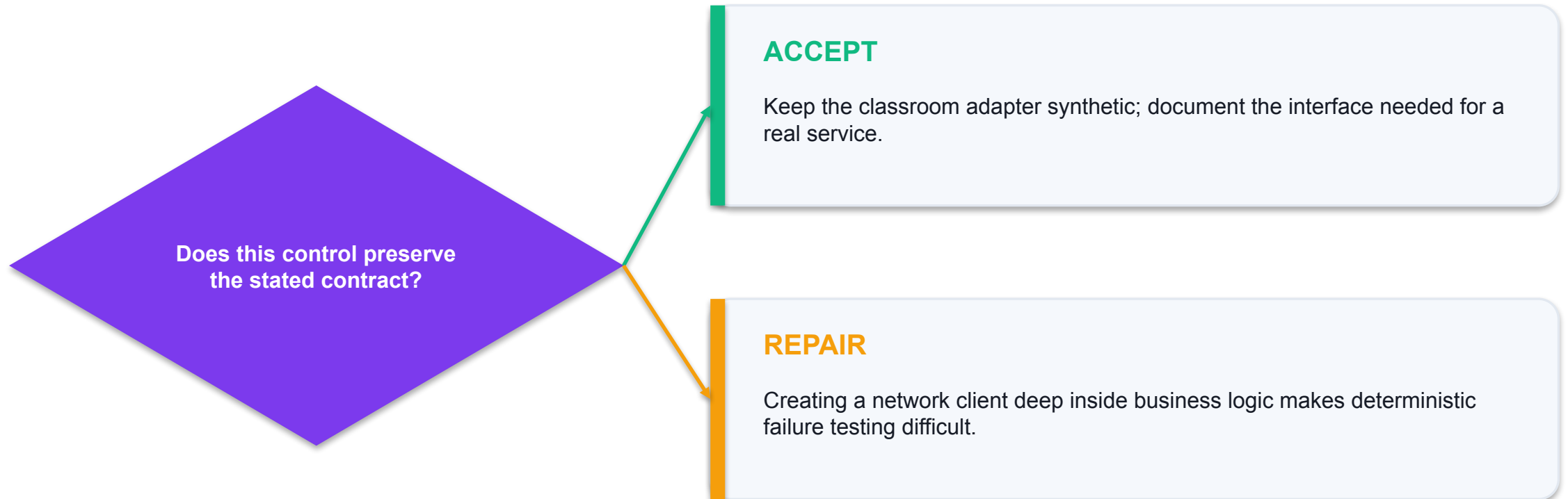
DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K3 / A3; apply to Activities 5–6.

Composition injects interchangeable services — design decision



Observable evidence: Dependencies are visible in construction and can be replaced by local mocks.

An interface is a caller-visible contract

Topic 3 · concrete C# mechanism and observable result

```
interface IGradable {  
    decimal CalculateAverage();  
    string GetGradeBand();  
}  
// Process both policies through IGradable.
```

OBSERVATION

A caller can use either student without knowing its concrete class.

INPUT / TYPE

```
interface IGradable {
```

DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K3 / A3; apply to Activities 5–6.

An interface is a caller-visible contract — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	A caller can use either student without knowing its concrete class.	Check the stated input/type and observed result.
Failure path	A shared method name without a common contract leaves the caller tied to concrete types.	Reproduce the smallest input that exposes this defect.
Design control	Test substitutability through the interface and state input/error guarantees.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Controls map to data and behaviour

Topic 3 · concrete C# mechanism and observable result

```
// Numeric input -> AddGrade(value)
// Student selector -> Student.Name
// Results grid -> average and band
// Confirmation -> successful model change
only
```

OBSERVATION

Each control has a specific property, rule and output responsibility.

INPUT / TYPE

// Numeric input -> AddGrade(value)

DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K3 / A3; apply to Activities 5–6.

Controls map to data and behaviour — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	Each control has a specific property, rule and output responsibility.	Check the stated input/type and observed result.
Failure path	A visually suitable control still needs range checks and error feedback.	Reproduce the smallest input that exposes this defect.
Design control	Choose controls by valid input range, allowed choices, readability and accessibility.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Catalogue classes, constructors and encapsulated properties

ACTIVITY 05

LO3 · K3 · A3

Administrators add product records. Product constructors enforce invariants and the catalogue owns its collection. Categories are typed; invalid records cannot enter the catalogue.

YOU'LL BUILD

Runnable C# console component,
recorded tests and a reviewed design
decision

TOOLCHAIN

.NET 10 SDK · C# · local synthetic data

DONE WHEN

An invalid name, price, stock or category
is rejected.

Activity 05 — vibe-coding prompt

Act as a C# reviewer. Work only in this .NET 10 console project. Read the synthetic specification: Administrators add product records. Product constructors enforce invariants and the catalogue owns its collection. Categories are typed; invalid records cannot enter the catalogue. Identify the input/output contract, invariants and error path. Return a short design and three test cases before proposing code. No new dependencies, network calls or file deletion.

INPUT

Synthetic specification + current source.

CONSTRAINT

C# / .NET 10; no new dependencies.

REVIEW

Minimal diff + concrete test cases.

EVIDENCE

Human explanation + actual execution.

Activity 05 — review and evidence prompts

REVIEW PROMPT

Review my Program.cs against requirements.md. Explain each defect using a concrete input and expected versus actual behaviour. Return the smallest C# diff and tests. Do not invent successful test results. If uncertain about an API, point to current Microsoft documentation.

EVIDENCE PROMPT

Using only my supplied actual build/run output, help me summarize one requirement-to-code-to-test link. Label any assumption. I must explain and approve the change myself.

Activity 05 — executable evidence

The baseline is supplied in this activity folder; full procedures are in the LG.

```
var catalogue = new ProductCatalogue();
catalogue.Add(new Product(
    "Keyboard", 49.90m, 10,
    ProductCategory.Electronics));
// Constructor enforces the product invariant.
// Catalogue count is now 1.
```

EXPECTED OUTPUT

Added Keyboard; catalogue count: 1
Keyboard | Electronics | 49.90 | stock 10

INVARIANT

An invalid name, price, stock or category is rejected.

FAILURE CHECK

The collection is exposed read-only and stock changes through a method.

REVIEW TARGET

Add stock adjustment checks for valid/invalid deltas.

Activity 05 — acceptance evidence

✓ Expected result

An invalid name, price, stock or category is rejected.
The collection is exposed read-only and stock changes through a method.
Three named categories are available.
Valid add increments the count and displays confirmation.

IF IT DOESN'T WORK

Compiler failure

Inspect the first C# error and the project target.

Wrong result

Compare the actual input with the named business invariant.

AI mismatch

Reject an unverified API or out-of-scope dependency change.

Interfaces, inheritance and polymorphic grade controls

ACTIVITY 06

LO3 · K3 · A3

A school needs extensible grade classification. A base Student owns validated grades; GradedStudent and HonoursStudent implement a shared IGradable contract and demonstrate virtual dispatch.

YOU'LL BUILD

Runnable C# console component,
recorded tests and a reviewed design
decision

TOOLCHAIN

.NET 10 SDK · C# · local synthetic data

DONE WHEN

All grades are within 0–100.

Activity 06 — vibe-coding prompt

Act as a C# reviewer. Work only in this .NET 10 console project. Read the synthetic specification: A school needs extensible grade classification. A base Student owns validated grades; GradedStudent and HonoursStudent implement a shared IGradable contract and demonstrate virtual dispatch. Identify the input/output contract, invariants and error path. Return a short design and three test cases before proposing code. No new dependencies, network calls or file deletion.

INPUT

Synthetic specification + current source.

CONSTRAINT

C# / .NET 10; no new dependencies.

REVIEW

Minimal diff + concrete test cases.

EVIDENCE

Human explanation + actual execution.

Activity 06 — review and evidence prompts

REVIEW PROMPT

Review my Program.cs against requirements.md. Explain each defect using a concrete input and expected versus actual behaviour. Return the smallest C# diff and tests. Do not invent successful test results. If uncertain about an API, point to current Microsoft documentation.

EVIDENCE PROMPT

Using only my supplied actual build/run output, help me summarize one requirement-to-code-to-test link. Label any assumption. I must explain and approve the change myself.

Activity 06 — executable evidence

The baseline is supplied in this activity folder; full procedures are in the LG.

```
IGradable student = honours;
Console.WriteLine(student.GetGradeBand());
// Grades 90 + 80 => mean 85.
// Standard: B; Honours: A.
// The interface call dispatches to the
// override.
```

EXPECTED OUTPUT

Mean 85.00; band B
Mean 85.00; band A

INVARIANT

All grades are within 0–100.

FAILURE CHECK

Empty grade collection fails with a meaningful error.

REVIEW TARGET

Add 89/90 and 84/85 boundary cases for the two policies.

Activity 06 — acceptance evidence

✓ Expected result

All grades are within 0–100.
Empty grade collection fails with a meaningful error.
The same mean 85 is B for standard and A for honours.
Both students are processed through IGradable.

IF IT DOESN'T WORK

Compiler failure

Inspect the first C# error and the project target.

Wrong result

Compare the actual input with the named business invariant.

AI mismatch

Reject an unverified API or out-of-scope dependency change.

TOPIC 4 · K4 / A4

Functionality and interoperability

Examine the interoperability and functionality of C# software components.

Struct construction and default values

Topic 4 · concrete C# mechanism and observable result

```
readonly struct Measurement {  
    public int Value { get; }  
    public Measurement() { Value=10; }  
}  
// new Measurement().Value -> 10  
// default(Measurement).Value -> 0
```

OBSERVATION

Modern C# allows a public parameterless struct constructor; default still zero-initializes.

INPUT / TYPE

readonly struct Measurement {

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K4 / A4; apply to Activities 7–8.

Struct construction and default values — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	Modern C# allows a public parameterless struct constructor; default still zero-initializes.	Check the stated input/type and observed result.
Failure path	Assuming default invokes the custom constructor gives an invalid invariant.	Reproduce the smallest input that exposes this defect.
Design control	Use structs for small value semantics and design for valid default state.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Enums need validation at external boundaries

Topic 4 · concrete C# mechanism and observable result

```
enum Category { Electronics, Clothing, Food }  
var category = (Category)99;  
Console.WriteLine(Enum.IsDefined(category)); /  
/ False
```

OBSERVATION

Casting a number can produce an unnamed enum value.

INPUT / TYPE

```
enum Category { Electronics, Clothing, Food }
```

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K4 / A4; apply to Activities 7–8.

Enums need validation at external boundaries — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	Casting a number can produce an unnamed enum value.	Check the stated input/type and observed result.
Failure path	An enum declaration alone does not reject invalid imported numeric values.	Reproduce the smallest input that exposes this defect.
Design control	Validate parsed enum values and document how names/numbers are exchanged.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Regex validates format, not domain semantics

Topic 4 · concrete C# mechanism and observable result

```
using System.Text.RegularExpressions;
bool ok = Regex.IsMatch("SKU-001", @"^ASKU-[0-9]{3}$",
    RegexOptions.CultureInvariant,
    TimeSpan.FromMilliseconds(100));
```

OBSERVATION

anchors require the entire SKU; timeout bounds the match work.

INPUT / TYPE

using System.Text.RegularExpressions;

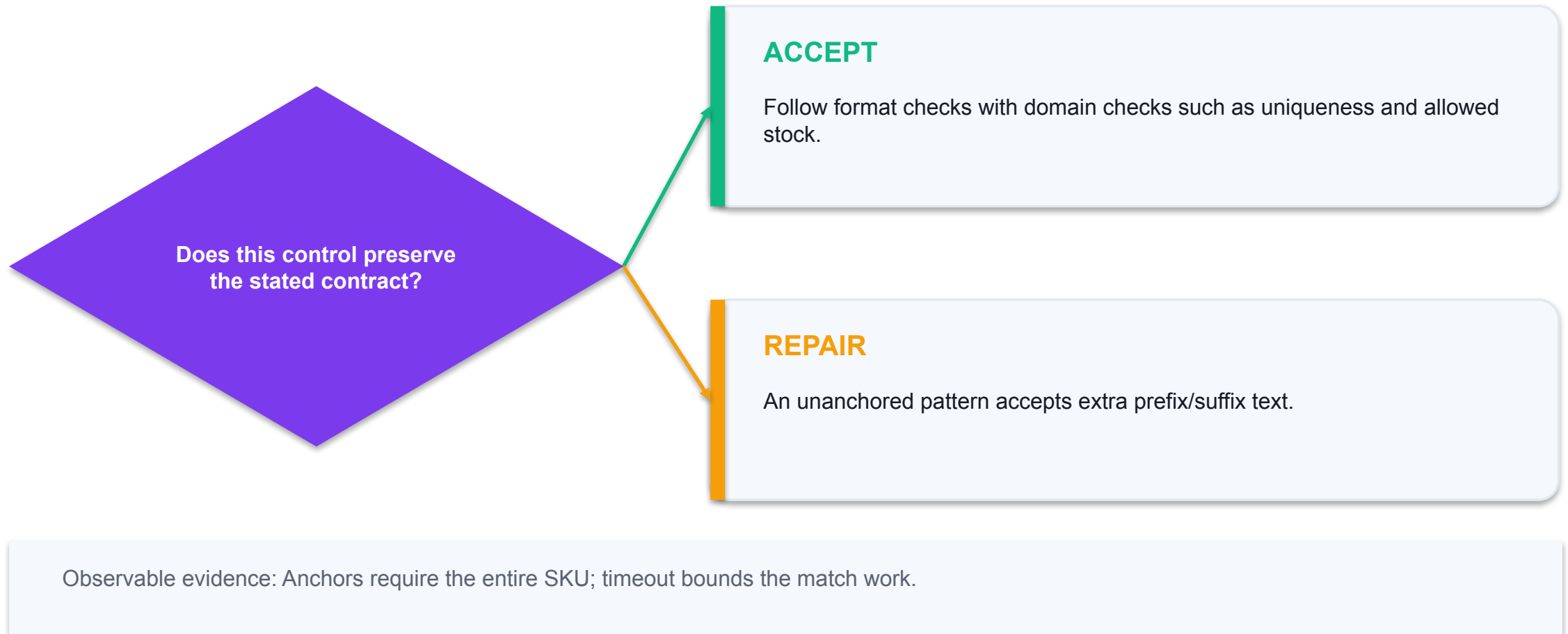
DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K4 / A4; apply to Activities 7–8.

Regex validates format, not domain semantics — design decision



Catch specific exceptions at a meaningful boundary

Topic 4 · concrete C# mechanism and observable result

```
try { int quantity = int.Parse("ten"); }  
catch(FormatException ex) {  
    Console.WriteLine($"Invalid quantity:  
{ex.GetType().Name}");  
}  
finally { /* release owned resources if needed  
*/ }
```

OBSERVATION

The format failure is handled while unrelated failures remain visible.

INPUT / TYPE

```
try { int quantity = int.Parse("ten"); }
```

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K4 / A4; apply to Activities 7–8.

Catch specific exceptions at a meaningful boundary — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	The format failure is handled while unrelated failures remain visible.	Check the stated input/type and observed result.
Failure path	A broad empty catch suppresses defects and may claim success incorrectly.	Reproduce the smallest input that exposes this defect.
Design control	Use TryParse for normal bad input; use catch for exceptional failures.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Custom exception and rethrow retain diagnostic context

Topic 4 · concrete C# mechanism and observable result

```
sealed class StockException(string message) :  
    Exception(message);  
// Within a catch block:  
// throw; preserves the original stack.  
// throw ex; restarts the stack at this line.
```

OBSERVATION

A named domain exception distinguishes an invariant failure.

INPUT / TYPE

```
sealed class StockException(string message) :  
    Exception(message);
```

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K4 / A4; apply to Activities 7–8.

Custom exception and rethrow retain diagnostic context — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	A named domain exception distinguishes an invariant failure.	Check the stated input/type and observed result.
Failure path	Replacing the original exception without context hides the failing call chain.	Reproduce the smallest input that exposes this defect.
Design control	Keep useful context and avoid sensitive data in exception messages.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Files and streams define an I/O contract

Topic 4 · concrete C# mechanism and observable result

```
using var stream = new FileStream("stock.txt",  
    FileMode.Create, FileAccess.Write);  
using var writer = new StreamWriter(stream);  
writer.WriteLine("SKU-001,10");
```

OBSERVATION

The writer encodes text over a byte stream and disposal releases handles.

INPUT / TYPE

```
using var stream = new FileStream("stock.txt",  
    FileMode.Create, FileAccess.Write);
```

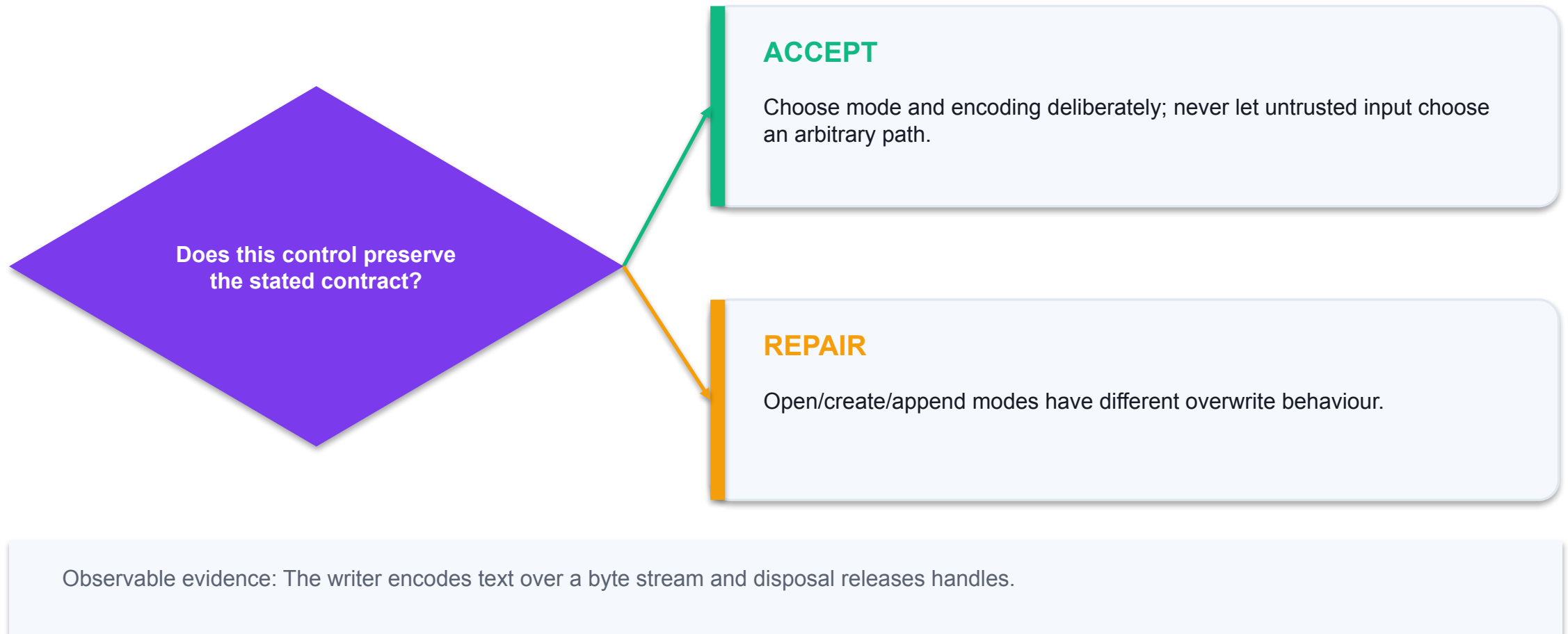
DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K4 / A4; apply to Activities 7–8.

Files and streams define an I/O contract — design decision



State transitions gate notifications

Topic 4 · concrete C# mechanism and observable result

```
// Pending -> PaymentProcessing
// success -> PaymentSuccessful -> Confirmed
-> notify
// reject/exception -> PaymentFailed -> no
notify
// cancellation -> Cancelled -> no
payment/notify
```

OBSERVATION

The outcome of one component controls what the next component may do.

INPUT / TYPE

// Pending -> PaymentProcessing

DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K4 / A4; apply to Activities 7–8.

State transitions gate notifications — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	The outcome of one component controls what the next component may do.	Check the stated input/type and observed result.
Failure path	Notifying before checking payment produces a false confirmation.	Reproduce the smallest input that exposes this defect.
Design control	Use explicit states and test every failure/cancellation path.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Interoperability needs a data exchange contract

Topic 4 · concrete C# mechanism and observable result

```
sealed record ProductDto(string Sku,string
Name,int Quantity);
// JSON:
{"Sku":"SKU-001","Name":"Keyboard","Quantity":
10}
// Quantity is a number, not "ten".
```

OBSERVATION

Names, types and allowed values are agreed by producer and consumer.

INPUT / TYPE

sealed record ProductDto(string Sku,string Name,int Quantity);

DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K4 / A4; apply to Activities 7–8.

Interoperability needs a data exchange contract — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	Names, types and allowed values are agreed by producer and consumer.	Check the stated input/type and observed result.
Failure path	Successful transport with a mismatched schema is still an integration failure.	Reproduce the smallest input that exposes this defect.
Design control	Test round-trip equality, missing values, invalid types and version changes.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

JSON success does not prove a valid domain object

Topic 4 · concrete C# mechanism and observable result

```
var p =  
JsonSerializer.Deserialize<ProductDto>(json);  
// Then validate p != null, SKU format,  
nonblank name,  
// and Quantity >= 0 before adding to  
catalogue.
```

OBSERVATION

Deserialization and business validation are distinct acceptance stages.

INPUT / TYPE

```
var p = JsonSerializer.Deserialize<ProductDto>(json);
```

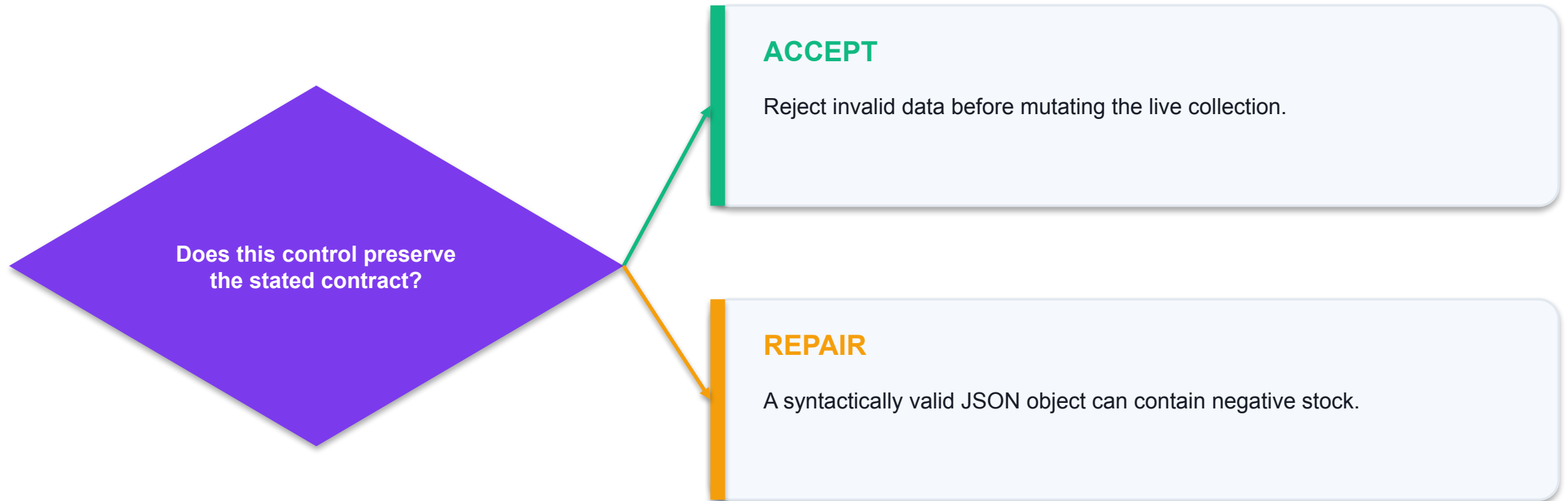
DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K4 / A4; apply to Activities 7–8.

JSON success does not prove a valid domain object — design decision



Observable evidence: Deserialization and business validation are distinct acceptance stages.

Logs link failures across components

Topic 4 · concrete C# mechanism and observable result

```
// ORDER-004: PaymentProcessing  
// ORDER-004: payment exception  
// Result: PaymentFailed; notified False
```

OBSERVATION

The correlation ID connects the payment failure to its resulting order state.

INPUT / TYPE

// ORDER-004: PaymentProcessing

DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K4 / A4; apply to Activities 7–8.

Logs link failures across components — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	The correlation ID connects the payment failure to its resulting order state.	Check the stated input/type and observed result.
Failure path	Logging only error text without an ID makes concurrent records hard to connect.	Reproduce the smallest input that exposes this defect.
Design control	Record identity, stage and outcome without exposing credentials or personal data.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Functionality indicators have an observable definition

Topic 4 · concrete C# mechanism and observable result

```
// Success rate = passed cases / attempted cases.  
// 4 correct paths / 4 exercised paths = 100%.  
// This is test-suite evidence, not a production SLA.
```

OBSERVATION

The numerator/denominator and exercised paths define what the figure proves.

INPUT / TYPE

// Success rate = passed cases / attempted cases.

DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K4 / A4; apply to Activities 7–8.

Functionality indicators have an observable definition — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	The numerator/denominator and exercised paths define what the figure proves.	Check the stated input/type and observed result.
Failure path	A single happy-path pass cannot establish all-path correctness.	Reproduce the smallest input that exposes this defect.
Design control	Report expected/actual outputs, error rate and timing under a stated workload.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Resource lifetime is part of interoperability

Topic 4 · concrete C# mechanism and observable result

```
using(var writer = new StreamWriter(path))  
  
writer.Write(JsonSerializer.Serialize(item));  
var reloaded = Import(File.ReadAllText(path));
```

OBSERVATION

Disposing the writer flushes/closes the file before the next component reads it.

INPUT / TYPE

using(var writer = new StreamWriter(path))

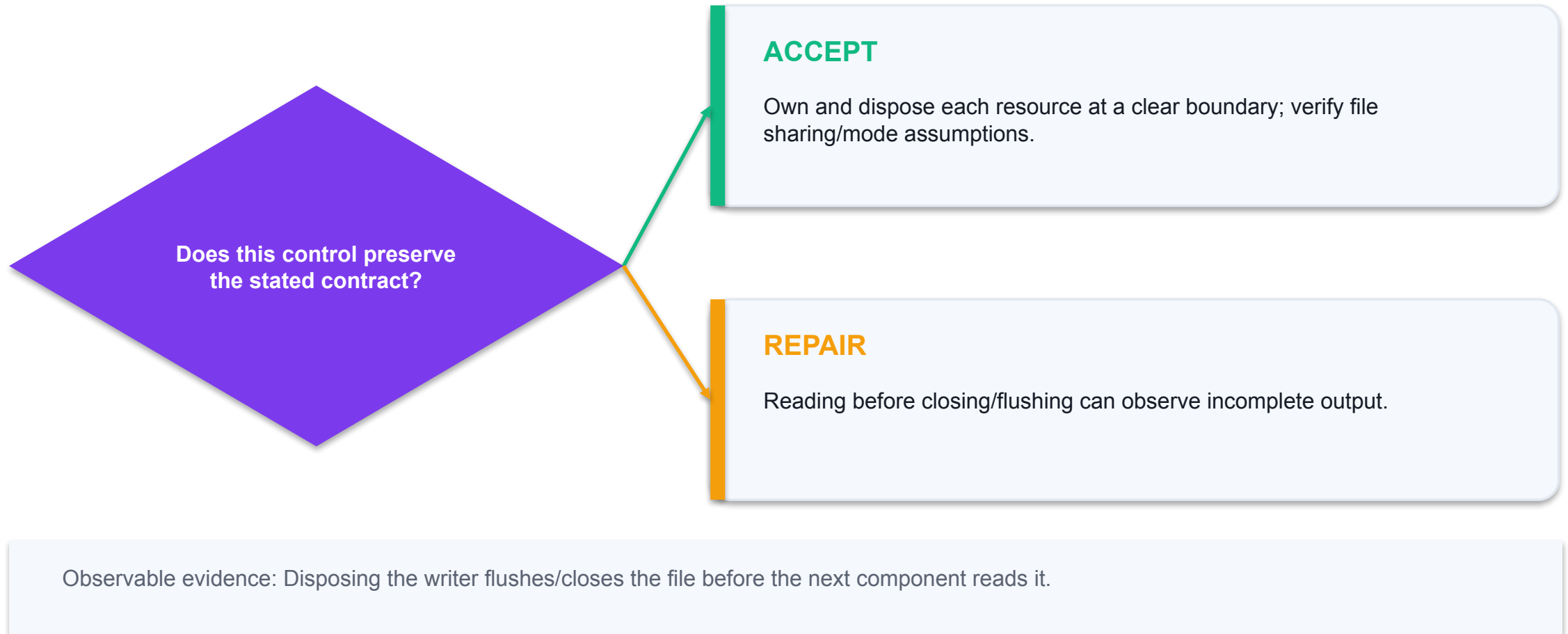
DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K4 / A4; apply to Activities 7–8.

Resource lifetime is part of interoperability — design decision



Repeated calls require a separate idempotency design

Topic 4 · concrete C# mechanism and observable result

```
// First Process(ORDER-001) -> notification.  
// Second Process(ORDER-001) currently repeats  
it.  
// Proposed guard: completed order-ID record.
```

OBSERVATION

A single-call test does not prove safe retries.

INPUT / TYPE

// First Process(ORDER-001) -> notification.

DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K4 / A4; apply to Activities 7–8.

Repeated calls require a separate idempotency design — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	A single-call test does not prove safe retries.	Check the stated input/type and observed result.
Failure path	A transient retry can duplicate an external effect.	Reproduce the smallest input that exposes this defect.
Design control	Document the sample limitation and test deduplication before claiming production readiness.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Order states, payment failure and notification guards

ACTIVITY 07

LO4 · K4 · A4

A food-delivery processor must never confirm a failed payment or notify a cancelled order. Mock services exercise success, rejection and thrown exceptions; status transitions are written to a local log.

YOU'LL BUILD

Runnable C# console component,
recorded tests and a reviewed design
decision

TOOLCHAIN

.NET 10 SDK · C# · local synthetic data

DONE WHEN

Success confirms and notifies exactly
once in a single call.

Activity 07 — vibe-coding prompt

Act as a C# reviewer. Work only in this .NET 10 console project. Read the synthetic specification: A food-delivery processor must never confirm a failed payment or notify a cancelled order. Mock services exercise success, rejection and thrown exceptions; status transitions are written to a local log. Identify the input/output contract, invariants and error path. Return a short design and three test cases before proposing code. No new dependencies, network calls or file deletion.

INPUT

Synthetic specification + current source.

CONSTRAINT

C# / .NET 10; no new dependencies.

REVIEW

Minimal diff + concrete test cases.

EVIDENCE

Human explanation + actual execution.

Activity 07 — review and evidence prompts

REVIEW PROMPT

Review my Program.cs against requirements.md. Explain each defect using a concrete input and expected versus actual behaviour. Return the smallest C# diff and tests. Do not invent successful test results. If uncertain about an API, point to current Microsoft documentation.

EVIDENCE PROMPT

Using only my supplied actual build/run output, help me summarize one requirement-to-code-to-test link. Label any assumption. I must explain and approve the change myself.

Activity 07 — executable evidence

The baseline is supplied in this activity folder; full procedures are in the LG.

```
var ok = processor.Process(
    "ORDER-001", true, false, false);
var fail = processor.Process(
    "ORDER-002", false, false, false);
// ok: Confirmed + notified.
// fail: PaymentFailed + no notification.
```

EXPECTED OUTPUT

ORDER-001: Confirmed; notified True
 ORDER-002: PaymentFailed; notified False
 Log: output/order-log.txt

INVARIANT

Success confirms and notifies exactly once in a single call.

FAILURE CHECK

Rejected or thrown payments fail without notification.

REVIEW TARGET

Read the local event log and build a state transition table.

Activity 07 — acceptance evidence

✓ Expected result

Success confirms and notifies exactly once in a single call.
Rejected or thrown payments fail without notification.
Cancelled orders bypass payment and notification.
Log contains the order ID and failure path.

IF IT DOESN'T WORK

Compiler failure

Inspect the first C# error and the project target.

Wrong result

Compare the actual input with the named business invariant.

AI mismatch

Reject an unverified API or out-of-scope dependency change.

JSON contracts, enum parsing, regex and file lifetime

ACTIVITY 08

LO4 · K4 · A4

A warehouse imports synthetic product data using a documented JSON contract. Reject mismatched types and negative stock, preserve UTF-8 names and prove export/import round-trip interoperability.

YOU'LL BUILD

Runnable C# console component,
recorded tests and a reviewed design
decision

TOOLCHAIN

.NET 10 SDK · C# · local synthetic data

DONE WHEN

A valid record survives JSON
export/import unchanged.

Activity 08 — vibe-coding prompt

Act as a C# reviewer. Work only in this .NET 10 console project. Read the synthetic specification: A warehouse imports synthetic product data using a documented JSON contract. Reject mismatched types and negative stock, preserve UTF-8 names and prove export/import round-trip interoperability. Identify the input/output contract, invariants and error path. Return a short design and three test cases before proposing code. No new dependencies, network calls or file deletion.

INPUT

Synthetic specification + current source.

CONSTRAINT

C# / .NET 10; no new dependencies.

REVIEW

Minimal diff + concrete test cases.

EVIDENCE

Human explanation + actual execution.

Activity 08 — review and evidence prompts

REVIEW PROMPT

Review my Program.cs against requirements.md. Explain each defect using a concrete input and expected versus actual behaviour. Return the smallest C# diff and tests. Do not invent successful test results. If uncertain about an API, point to current Microsoft documentation.

EVIDENCE PROMPT

Using only my supplied actual build/run output, help me summarize one requirement-to-code-to-test link. Label any assumption. I must explain and approve the change myself.

Activity 08 — executable evidence

The baseline is supplied in this activity folder; full procedures are in the LG.

```
var item = Import(sample);
var reloaded = Import(File.ReadAllText(path));
// Import validates SKU, name and quantity.
// Unicode survives the UTF-8 round trip.
// Wrong JSON types are rejected.
```

EXPECTED OUTPUT

Imported SKU-001; stock 10
Round-trip equal: True
Export: output/product.json

INVARIANT

A valid record survives JSON export/import unchanged.

FAILURE CHECK

Quantity is a JSON integer, not text; stock cannot be negative.

REVIEW TARGET

Create a contract table with field, type, required value, valid example and failure.

Activity 08 — acceptance evidence

✓ Expected result

A valid record survives JSON export/import unchanged.
Quantity is a JSON integer, not text; stock cannot be negative.
SKU uses exactly SKU- plus three digits.
UTF-8 names and zero stock are preserved.

IF IT DOESN'T WORK

Compiler failure

Inspect the first C# error and the project target.

Wrong result

Compare the actual input with the named business invariant.

AI mismatch

Reject an unverified API or out-of-scope dependency change.

TOPIC 5 · K5 / A5

Design documentation and handover

Generate C# design documentation aligned with user specifications.

Namespaces and using resolve symbols

Topic 5 · concrete C# mechanism and observable result

```
using System.Text.Json;  
namespace Warehouse.Design;  
public sealed record ProductDto(string Sku,int  
Quantity);
```

OBSERVATION

Namespaces organize names; using makes names available without a qualified prefix.

INPUT / TYPE

using System.Text.Json;

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K5 / A5; apply to Activities 9–10.

Namespaces and using resolve symbols — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	Namespaces organize names; using makes names available without a qualified prefix.	Check the stated input/type and observed result.
Failure path	Two imported types with the same name create ambiguity.	Reproduce the smallest input that exposes this defect.
Design control	Use qualified names or aliases; a namespace is not a deployment/security boundary.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Conditional compilation changes the built artifact

Topic 5 · concrete C# mechanism and observable result

```
#define DEMO
#if DEMO
Console.WriteLine("Synthetic local data");
#else
Console.WriteLine("Configured production
adapter");
#endif
```

OBSERVATION

Only the selected branch is compiled into the assembly.

INPUT / TYPE

#define DEMO

DESIGN CONTEXT

Legacy coverage updated for modern C#.

TRACE

K5 / A5; apply to Activities 9–10.

Conditional compilation changes the built artifact — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	Only the selected branch is compiled into the assembly.	Check the stated input/type and observed result.
Failure path	Testing only a debug branch leaves another compiled configuration unverified.	Reproduce the smallest input that exposes this defect.
Design control	Keep configuration explicit; #define does not create a normal runtime variable.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

XML comments describe the public API contract

Topic 5 · concrete C# mechanism and observable result

```
/// <summary>Charges 0.50 per overdue calendar
day.</summary>
/// <param name="due">Due date.</param>
/// <param name="returned">Actual return
date.</param>
/// <returns>Non-negative fine in
SGD.</returns>
public static decimal Calculate(DateOnly
due,DateOnly returned)
=> Math.Max(0,returned.DayNumber-
due.DayNumber)*0.50m;
```

OBSERVATION

Summary, parameter and return tags connect the API to the documented rule.

INPUT / TYPE

/// <summary>Charges 0.50 per overdue calendar day.</summary>

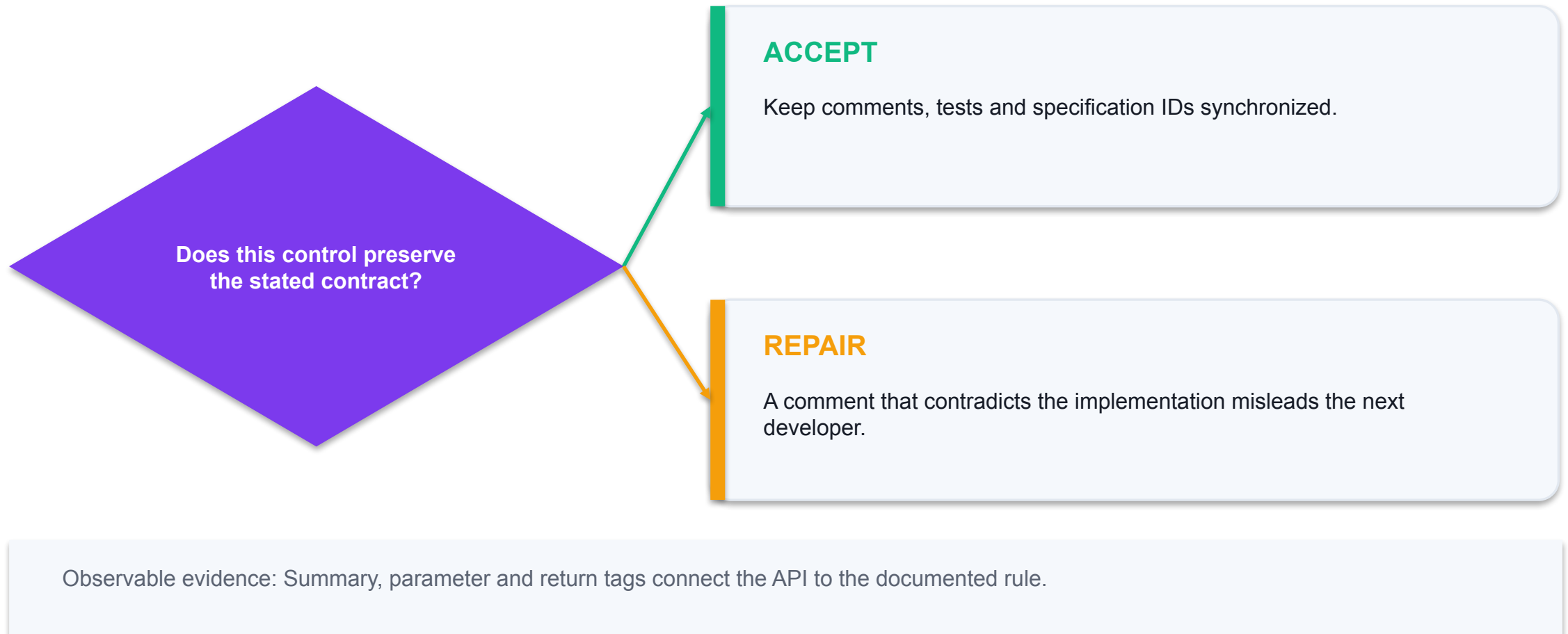
DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K5 / A5; apply to Activities 9–10.

XML comments describe the public API contract — design decision



Documentation is generated from a compiler setting

Topic 5 · concrete C# mechanism and observable result

```
<GenerateDocumentationFile>true</  
GenerateDocumentationFile>  
// Build emits Activity.xml beside the  
assembly.  
// CS1591 identifies undocumented public API  
when enabled.
```

OBSERVATION

The XML output is a real build artifact that can be inspected.

INPUT / TYPE

```
<GenerateDocumentationFile>true</  
GenerateDocumentationFile>
```

DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K5 / A5; apply to Activities 9–10.

Documentation is generated from a compiler setting — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	The XML output is a real build artifact that can be inspected.	Check the stated input/type and observed result.
Failure path	A checked-in comment alone does not prove documentation generation is configured.	Reproduce the smallest input that exposes this defect.
Design control	Inspect the emitted XML and document any deliberate warning policy.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Requirement traceability reaches executable evidence

Topic 5 · concrete C# mechanism and observable result

```
// R4 fine policy  
// -> FineCalculator.Calculate  
// -> due 10 Sep, return 13 Sep  
// -> expected 1.50, actual 1.50
```

OBSERVATION

A reviewer can follow one user requirement into a method and a concrete test.

INPUT / TYPE

// R4 fine policy

DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K5 / A5; apply to Activities 9–10.

Requirement traceability reaches executable evidence — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	A reviewer can follow one user requirement into a method and a concrete test.	Check the stated input/type and observed result.
Failure path	A mapping that ends at a class name cannot prove the required behaviour.	Reproduce the smallest input that exposes this defect.
Design control	Record requirement, method, test input, expected/actual output and evidence location.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Dependency notes identify real package boundaries

Topic 5 · concrete C# mechanism and observable result

```
// Target: .NET 10 (net10.0).  
// BCL: System.IO, System.Text.Json, Regex.  
// Third-party NuGet packages: none.  
// Optional editor/AI tools are not runtime  
dependencies.
```

OBSERVATION

The handover distinguishes shipped code dependencies from authoring tools.

INPUT / TYPE

// Target: .NET 10 (net10.0).

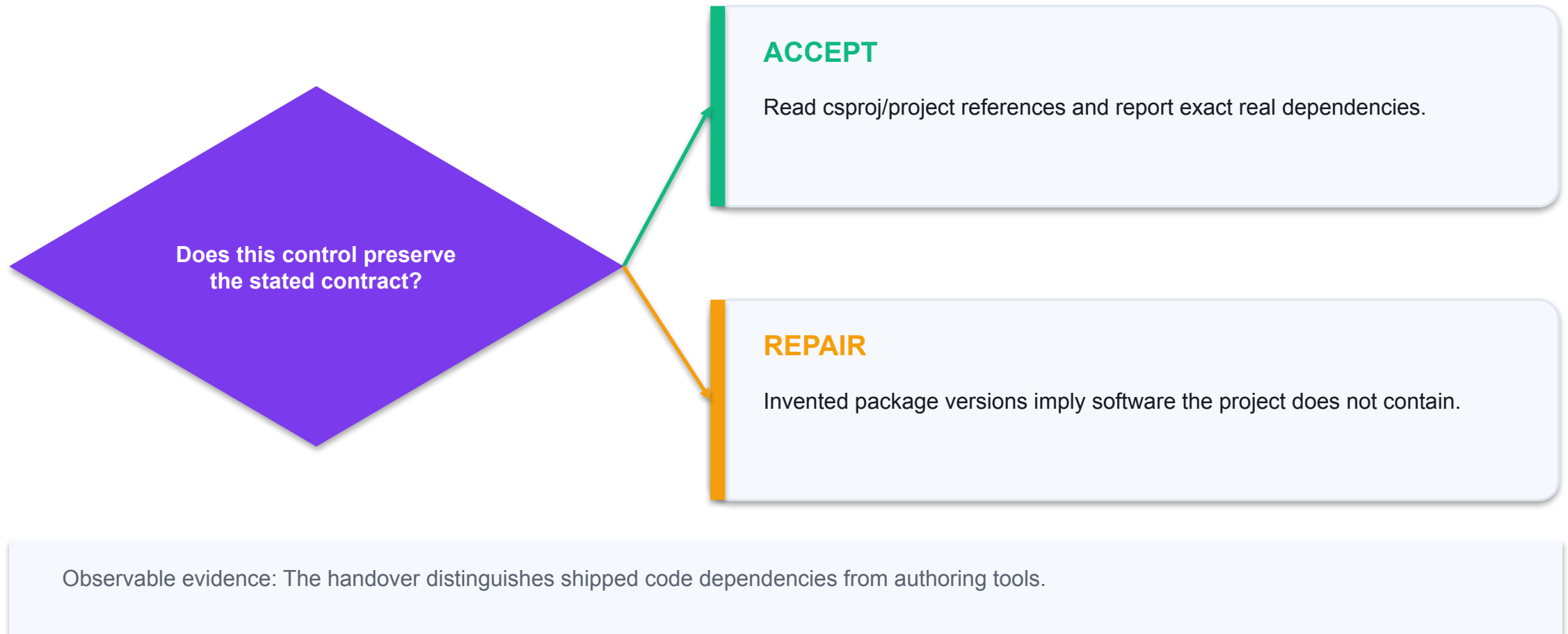
DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K5 / A5; apply to Activities 9–10.

Dependency notes identify real package boundaries — design decision



A design decision record documents the tradeoff

Topic 5 · concrete C# mechanism and observable result

```
// Decision: encapsulate stock behind  
Add/Remove.  
// Alternative: unrestricted public setter.  
// Reason: preserve nonnegative stock on every  
path.  
// Consequence: callers handle rejected  
operations.
```

OBSERVATION

The chosen control, alternative and consequences explain why the implementation has its shape.

INPUT / TYPE

// Decision: encapsulate stock behind Add/Remove.

DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K5 / A5; apply to Activities 9–10.

A design decision record documents the tradeoff — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	The chosen control, alternative and consequences explain why the implementation has its shape.	Check the stated input/type and observed result.
Failure path	Documenting only the chosen class omits the reason and migration limits.	Reproduce the smallest input that exposes this defect.
Design control	Tie decisions to requirements and revisit them when requirements change.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

A class library moves code across a project boundary

Topic 5 · concrete C# mechanism and observable result

```
<ProjectReference  
Include="../Stock.Core/Stock.Core.csproj" />  
// App calls public StockService from  
Stock.Core.  
// Internal types remain within their  
assembly.
```

OBSERVATION

The build resolves a reusable local component as a separate referenced assembly.

INPUT / TYPE

```
<ProjectReference Include="../Stock.Core/Stock.Core.csproj"  
/>
```

DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K5 / A5; apply to Activities 9–10.

A class library moves code across a project boundary — failure analysis

Evidence layer	Concrete signal	Review action
Correct behaviour	The build resolves a reusable local component as a separate referenced assembly.	Check the stated input/type and observed result.
Failure path	Copying the same class into multiple apps creates drifting implementations.	Reproduce the smallest input that exposes this defect.
Design control	Document the public API and rerun consumer tests after library changes.	Map the control to the specification and test the rejected path.

WHEN IT MATTERS Explain the mechanism before approving a proposed code change; record actual results rather than an AI claim.

Handover states practical limits

Topic 5 · concrete C# mechanism and observable result

```
// Sample: in-memory, single process, no user  
auth.  
// Data reset when application exits.  
// Future work: persistence, authorization,  
concurrency.  
// Each new feature requires a new  
contract/test.
```

OBSERVATION

The receiver can see what the working sample proves and what must be engineered next.

INPUT / TYPE

// Sample: in-memory, single process, no user auth.

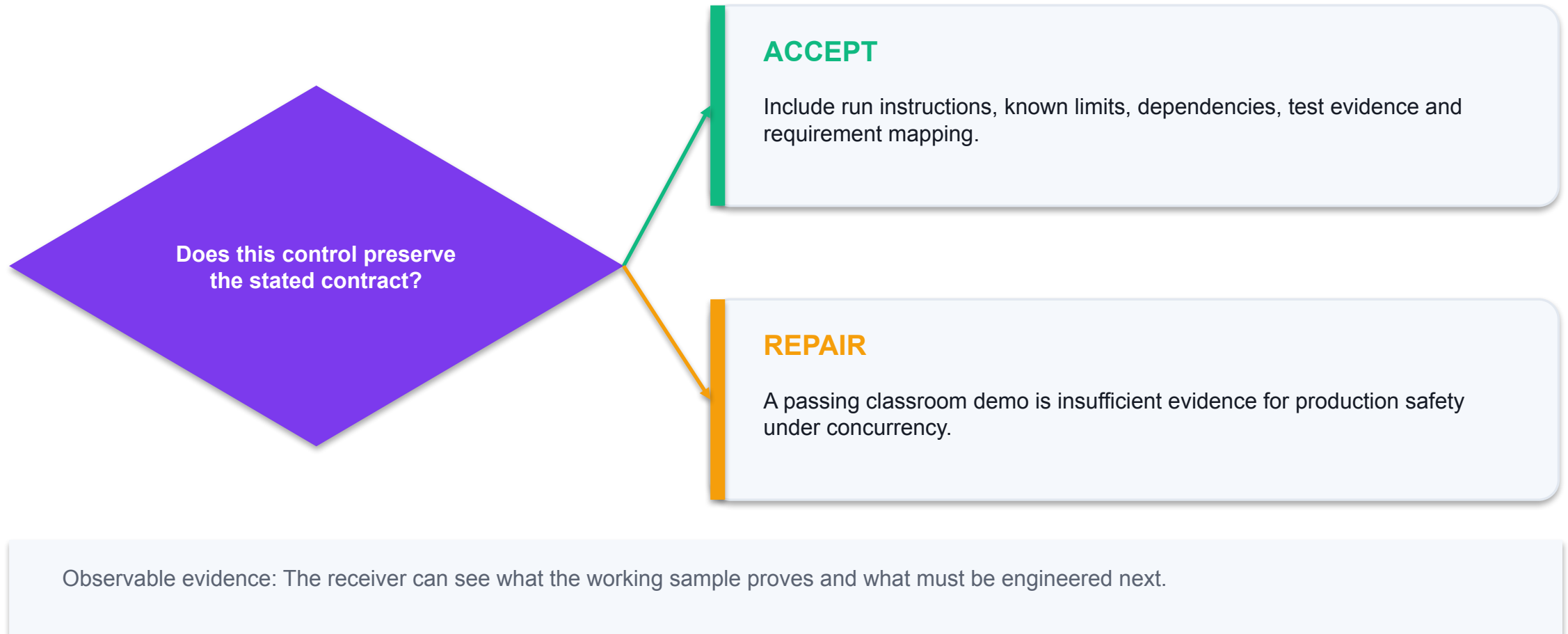
DESIGN CONTEXT

Additional design and evidence depth.

TRACE

K5 / A5; apply to Activities 9–10.

Handover states practical limits — design decision



Library methods, XML documentation and specification traceability

ACTIVITY 09

LO5 · K5 · A5

A council library requires book search, member registration, loan processing and a fine of 0.50 per overdue calendar day. Produce XML comments, dependency notes and an explicit requirement-to-method-to-test map.

YOU'LL BUILD

Runnable C# console component,
recorded tests and a reviewed design
decision

TOOLCHAIN

.NET 10 SDK · C# · local synthetic data

DONE WHEN

Book search is case-insensitive.

Activity 09 — vibe-coding prompt

Act as a C# reviewer. Work only in this .NET 10 console project. Read the synthetic specification: A council library requires book search, member registration, loan processing and a fine of 0.50 per overdue calendar day. Produce XML comments, dependency notes and an explicit requirement-to-method-to-test map. Identify the input/output contract, invariants and error path. Return a short design and three test cases before proposing code. No new dependencies, network calls or file deletion.

INPUT

Synthetic specification + current source.

CONSTRAINT

C# / .NET 10; no new dependencies.

REVIEW

Minimal diff + concrete test cases.

EVIDENCE

Human explanation + actual execution.

Activity 09 — review and evidence prompts

REVIEW PROMPT

Review my Program.cs against requirements.md. Explain each defect using a concrete input and expected versus actual behaviour. Return the smallest C# diff and tests. Do not invent successful test results. If uncertain about an API, point to current Microsoft documentation.

EVIDENCE PROMPT

Using only my supplied actual build/run output, help me summarize one requirement-to-code-to-test link. Label any assumption. I must explain and approve the change myself.

Activity 09 — executable evidence

The baseline is supplied in this activity folder; full procedures are in the LG.

```
library.RegisterMember("M001");
library.LoanBook("B001", "M001");
var fine = FineCalculator.Calculate(
    new DateOnly(2026, 9, 10),
    new DateOnly(2026, 9, 13));
// Three overdue days => SGD 1.50.
```

EXPECTED OUTPUT

Search matches: 1
Fine: 1.50
Report: output/design-report.txt

INVARIANT

Book search is case-insensitive.

FAILURE CHECK

Only registered members may loan an available book.

REVIEW TARGET

Build XML documentation and locate
bin/Debug/net10.0/Activity.xml.

Activity 09 — acceptance evidence

✓ Expected result

Book search is case-insensitive.
Only registered members may loan an available book.
Three overdue calendar days produce 1.50; early return produces zero.
Generated report maps R1–R4 to named public methods.

IF IT DOESN'T WORK

Compiler failure

Inspect the first C# error and the project target.

Wrong result

Compare the actual input with the named business invariant.

AI mismatch

Reject an unverified API or out-of-scope dependency change.

Documented stock component and handover evidence

ACTIVITY 10

LO5 · K5 · A5

A warehouse requires a reusable stock service that prevents negative stock, reports changes and can be traced to R-STOCK-01. Deliver a design decision, dependency inventory, public API comments and executable evidence.

YOU'LL BUILD

Runnable C# console component,
recorded tests and a reviewed design
decision

TOOLCHAIN

.NET 10 SDK · C# · local synthetic data

DONE WHEN

10 – 3 + 2 produces stock 9.

Activity 10 — vibe-coding prompt

Act as a C# reviewer. Work only in this .NET 10 console project. Read the synthetic specification: A warehouse requires a reusable stock service that prevents negative stock, reports changes and can be traced to R-STOCK-01. Deliver a design decision, dependency inventory, public API comments and executable evidence. Identify the input/output contract, invariants and error path. Return a short design and three test cases before proposing code. No new dependencies, network calls or file deletion.

INPUT

Synthetic specification + current source.

CONSTRAINT

C# / .NET 10; no new dependencies.

REVIEW

Minimal diff + concrete test cases.

EVIDENCE

Human explanation + actual execution.

Activity 10 — review and evidence prompts

REVIEW PROMPT

Review my Program.cs against requirements.md. Explain each defect using a concrete input and expected versus actual behaviour. Return the smallest C# diff and tests. Do not invent successful test results. If uncertain about an API, point to current Microsoft documentation.

EVIDENCE PROMPT

Using only my supplied actual build/run output, help me summarize one requirement-to-code-to-test link. Label any assumption. I must explain and approve the change myself.

Activity 10 — executable evidence

The baseline is supplied in this activity folder; full procedures are in the LG.

```
var stock = new StockService(10);
stock.Remove(3);
stock.Add(2);
// Count = 9.
// Remove(10) rejects the change.
// Rejection leaves Count at 9.
```

EXPECTED OUTPUT

Stock after remove/add: 9
Handover: output/handover.txt

INVARIANT

$10 - 3 + 2$ produces stock 9.

FAILURE CHECK

Overselling is rejected without changing existing stock.

REVIEW TARGET

Move StockService into a documented class library and add a project reference; rerun the same tests.

Activity 10 — acceptance evidence

✓ Expected result

10 – 3 + 2 produces stock 9.

Overselling is rejected without changing existing stock.

Negative and overflow mutations are rejected.

Handover states the contract, requirement trace, dependencies and limitations.

IF IT DOESN'T WORK

Compiler failure

Inspect the first C# error and the project target.

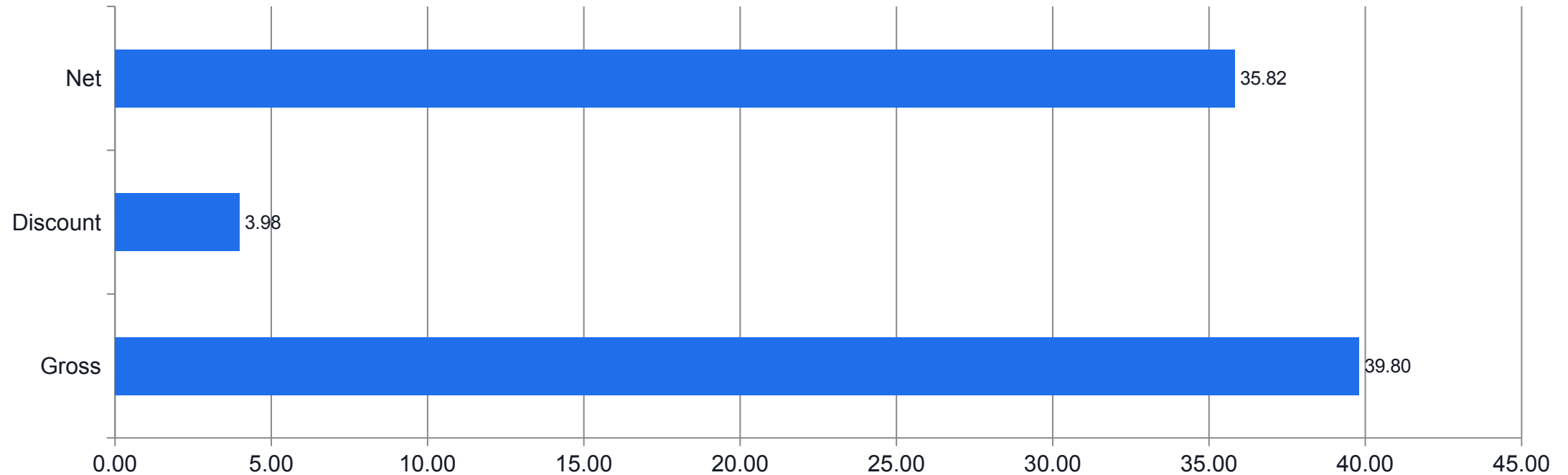
Wrong result

Compare the actual input with the named business invariant.

AI mismatch

Reject an unverified API or out-of-scope dependency change.

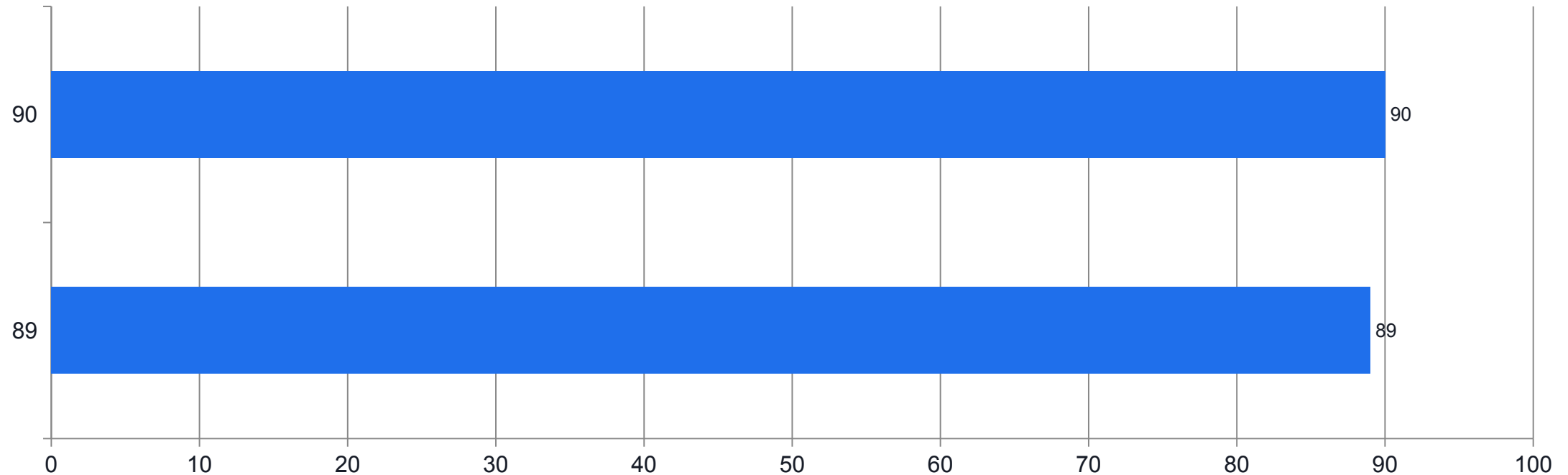
Receipt calculation — worked values



WHAT THE DATA SHOWS

$19.90 \times 2 = 39.80$; 10% discount = 3.98; final receipt = 35.82.

Grade boundary — a one-point difference

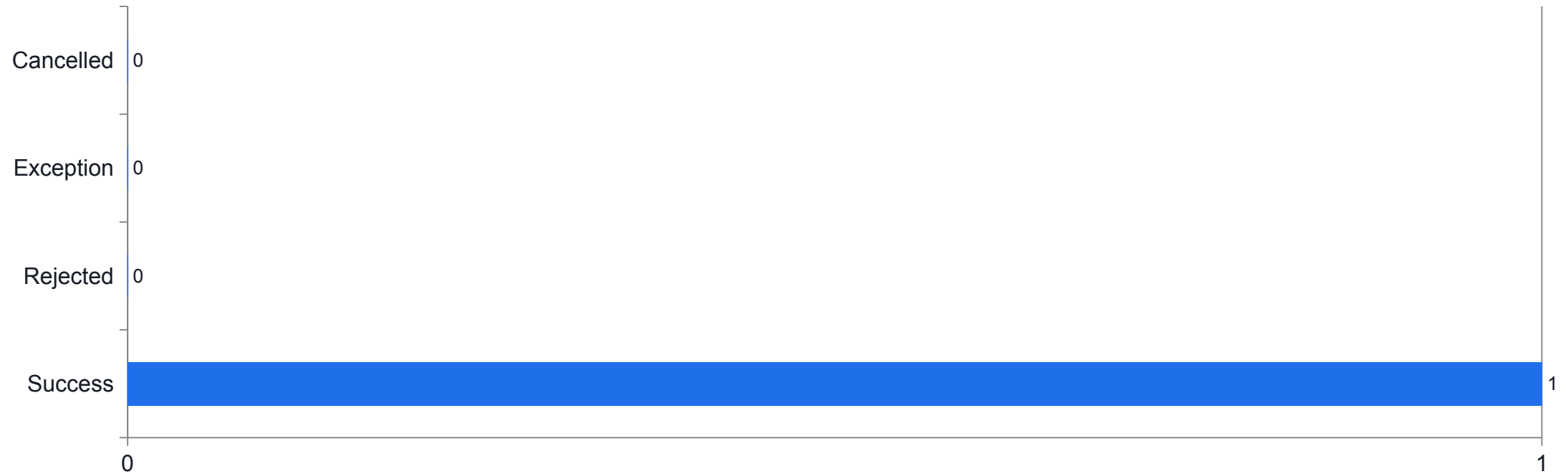


WHAT THE DATA SHOWS

Standard policy: 89 is B and 90 is A. The $\geq$ comparison is the deciding control.

EXPECTED SINGLE-CALL RESULTS

Order integration — expected notification count

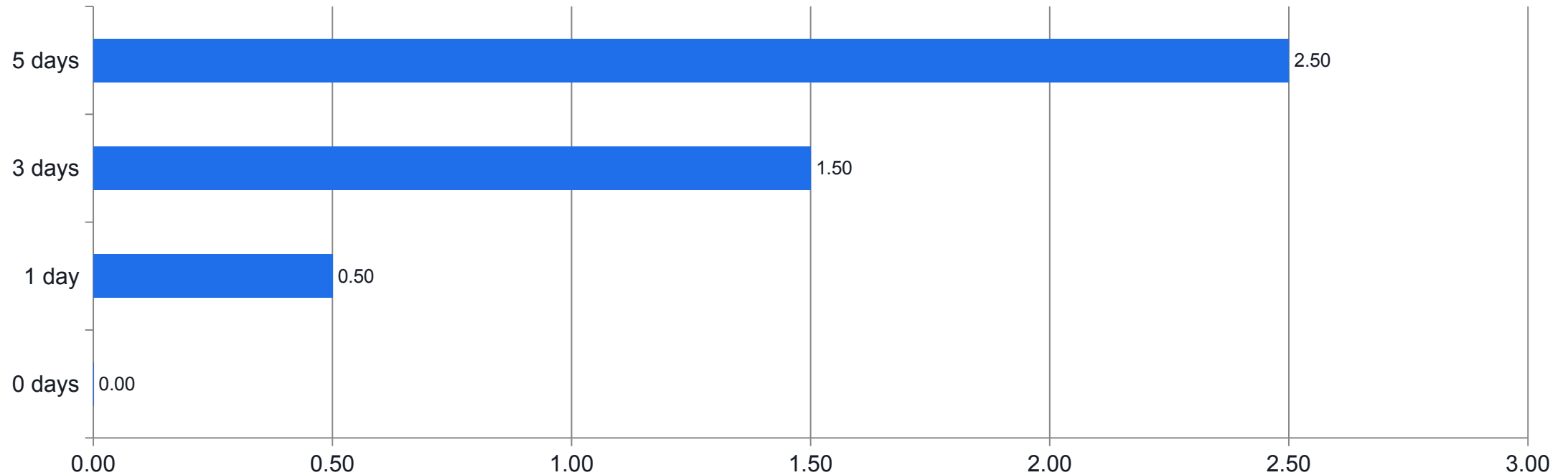


WHAT THE DATA SHOWS

Only the confirmed success path notifies. These are expected classroom cases, not production measurements.

WORKED POLICY VALUES

Library fine — calendar-day formula



WHAT THE DATA SHOWS

Fine = $\max(0, \text{returned.DayNumber} - \text{due.DayNumber}) \times 0.50$. Early returns never produce a negative fine.

What You Built

1

K1 / A1

Determine basic software components using C# methodologies to meet functional specifications.

2

K2 / A2

Apply C# methodologies and tools for software creation.

3

K3 / A3

Select essential C# controls and features to meet software design requirements.

4

K4 / A4

Examine the interoperability and functionality of C# software components.

5

K5 / A5

Generate C# design documentation aligned with user specifications.

Assessment

1

WA

5 SAQs · K1–K5 · 60 min.

2

PP

5 practical tasks · A1–A5 · 60 min.

3

Resources

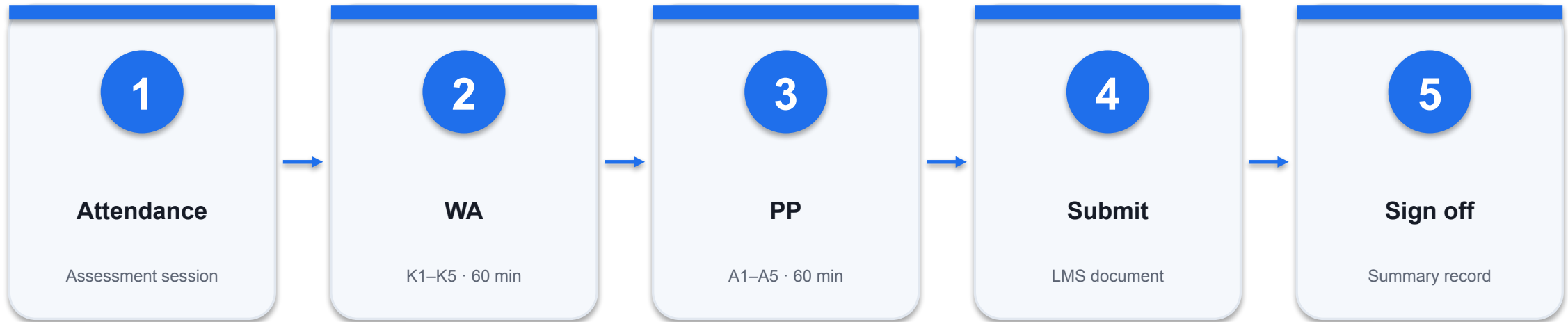
Open book with approved course resources.

4

Completion

Submit both documents on LMS and sign the summary record.

Assessment Flow



SIGN-OFF

Complete WA, PP, LMS submission, feedback and assessment summary record.

Digital Attendance (Mandatory)

1

AM / PM

Record both daily sessions using the current QR supplied by the trainer.

2

Assessment attendance

Record the assessment session before beginning the papers.

3

QR identity

Use the assigned attendance record and verify the submitted session.

4

Course portal

The trainer directs the current SSG/TRAQOM attendance workflow through the LMS.

TERTIARY INFOTECH ACADEMY

Thank You

enquiry@tertiaryinfotech.com · +65 6100 0613 · www.tertiarycourses.com.sg